DineDash Food Delivery Data Anaysis

1. Background

DineDash food delivery industry has seen rapid growth over the last decade, fueled by the increasing demand for convenience, improved delivery logistics, and the expansion of digital platforms. MealDash have revolutionized how food is ordered and delivered, creating new opportunities for restaurant owners and consumers alike.

However, as the industry continues to expand, food delivery platforms face increasing competition, customer expectations for faster and more personalized service, and challenges in optimizing their operations across a diverse and extensive menu offering.

> In this case study, we focus on the use of **big data and advanced analytics** to streamline operations, optimize delivery times, enhance customer experience, and improve menu offerings. By leveraging data processing platforms such as **Apache Spark** and **Databricks**, we can better understand customer preferences, predict demand, and offer personalized recommendations, ultimately driving more efficient and profitable business operations.

2. Challenges

Dinedash has to address several challenges to maintain competitive advantage and operational efficiency:

- **Data Fragmentation**: Different systems for orders, customer profiles, menus, and delivery logs lead to fragmented data that is difficult to analyze comprehensively.
- **Customer Segmentation**: Understanding customer preferences based on historical data to offer personalized promotions, menu recommendations, or loyalty rewards.
- **Order Prediction**: Predicting order volume in different regions at different times of the day to optimize delivery capacity and reduce wait times.
- **Menu Optimization**: Identifying the most popular and profitable menu items while suggesting item combinations to customers. Ensuring the menu is always updated in real-time for availability.
- **Order Fulfillment Delays**: Ensuring on-time deliveries and improving operational workflows, which are often impacted by external factors like weather, traffic, or restaurant delays.

- **Promotions and Discounts**: Identifying the most effective promotional offers and discounts that increase order value and drive repeat purchases without eroding profitability.

3. Business Needs

To stay competitive and meet customer demands, DineDash needs to address the following:

- **Data Integration**: Integrating data from different sources (customer orders, restaurant data, delivery times, etc.) to build a comprehensive view of customer behavior, restaurant performance, and delivery operations.
- **Real-Time Analytics**: Implementing a real-time data processing pipeline that analyzes orders as they come in, providing instant insights and enabling dynamic decision-making (e.g., dynamic pricing, offer recommendations).
- **Predictive Analytics**: Using machine learning to predict demand, customer preferences, delivery time, and possible bottlenecks in the system.

- **Optimization Algorithms**: Implementing algorithms that optimize the delivery process by calculating the best route, available delivery agents, and reducing food delivery time.
- **Personalization**: Offering personalized promotions, product recommendations, and even tailored menus based on past customer behavior, location, and preferences.
- **Performance Tracking**: Continuously monitoring key performance indicators (KPIs), such as customer satisfaction, order fulfillment rate, delivery time, and revenue per order.

4. Proposed Solution

The proposed solution involves setting up a **big data processing pipeline** using **Databricks** and **Apache Spark**, along with **machine learning models** to provide real-time insights and predictive analytics. The solution leverages:

- **Data Lake**: A centralized repository for storing all structured and unstructured data, including order details, customer information, menu data, and delivery logs.
- **ETL Pipelines**: Using **Delta Live Tables** (DLT) to manage the transformation of raw data into usable formats for analysis. The pipeline will clean, enrich, and structure the data for downstream analytics.
- **Real-Time Streaming**: Processing incoming orders in real-time using **structured streaming** to calculate metrics like order volume, delivery times, and detect any operational anomalies as they occur.
- **Analytics Dashboards**: Developing dashboards to visualize customer trends, restaurant performance, order patterns, and delivery efficiency.
- **Machine Learning Models**: Implementing models for demand forecasting, customer segmentation, and personalized recommendations to boost sales and improve customer experience.
- **Optimized Delivery Routing**: Using geographic data and algorithms to optimize delivery routes and reduce delivery times.

We will focus upon ETL Pipelines,Streaming Data analytics and Dashboards

5. Attributes

DineDash will track the following key attributes across different tables:

1. Customers

- customer_id: Unique identifier for each customer.
- name: Name of the customer.
- email: Email address of the customer.
- signup_date: Date customer signed up

2. Restaurants

- restaurant_id: Unique identifier for each restaurant.
- name: Name of the restaurant.
- cuisine: Type of cuisine (e.g., Italian, Chinese, Indian).
- location_id: Geographic location (latitude/longitude) of the restaurant

3. Menu Items

- item_id: Unique identifier for each menu item.
- restaurant_id: Foreign key referencing restaurants.
- item_name: Name of the menu item (e.g., "Margherita Pizza").
- price: Price of the menu item.

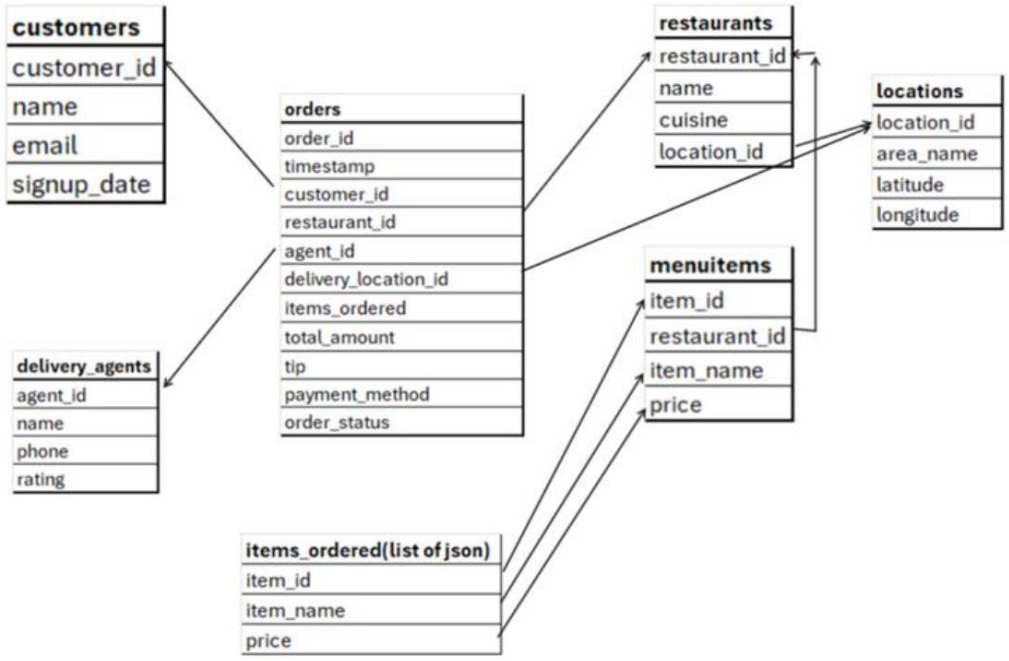
4. Orders

- order_id: Unique identifier for each order.
- timestamp: Time the order was placed.
- customer_id: Foreign key referencing customers.
- restaurant_id: Foreign key referencing restaurants.
- agent_id:
- delivery_location_id:
- items_ordered: List of items in the order (can be a list of menu_item_ids).
- Total_amount:
- Tip:
- payment_method: Payment method used (e.g., "Credit Card", "Cash").
- order_status: Status of the order (e.g., "Pending", "Delivered").

5. Delivery Agent

- agent_id: Unique identifier for each delivery.
- name:
- phone:
- rating:

Data model

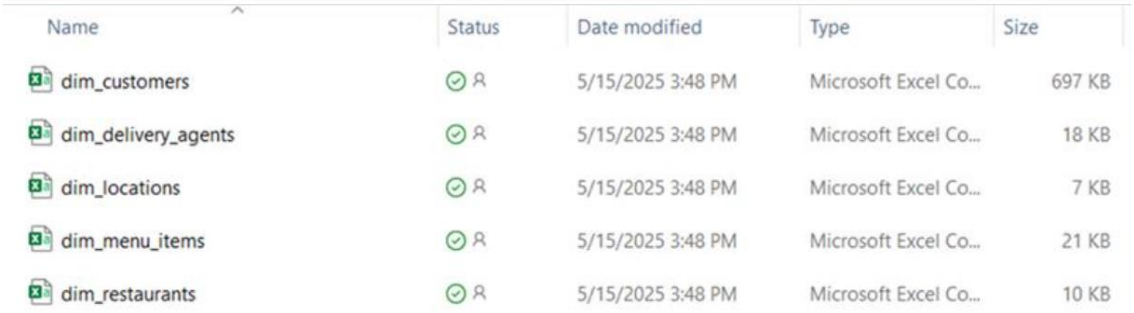


Datasets:

Orders dataset files:

Name	Status	Date modified	Type	Size
orders_2024_01		5/15/2025 3:48 PM	JSON Source File	2,522 KB
orders_2024_02		5/15/2025 3:48 PM	JSON Source File	2,438 KB
orders_2024_03		5/15/2025 3:48 PM	JSON Source File	2,143 KB
orders_2024_04		5/15/2025 3:48 PM	JSON Source File	3,425 KB
orders_2024_05		5/22/2025 7:45 PM	JSON Source File	2,364 KB
orders_2024_06		5/15/2025 3:48 PM	JSON Source File	3,428 KB
orders_2024_07		5/15/2025 3:48 PM	JSON Source File	2,769 KB
orders_2024_08		5/15/2025 3:48 PM	JSON Source File	3,265 KB
orders_2024_09		5/15/2025 3:48 PM	JSON Source File	2,421 KB
orders_2024_10		5/15/2025 3:48 PM	JSON Source File	3,307 KB
orders_2024_11		5/15/2025 3:49 PM	JSON Source File	2,145 KB
orders_2024_12		5/15/2025 3:49 PM	JSON Source File	2,221 KB

Other datasets files:



Name	Status	Date modified	Type	Size
dim_customers		5/15/2025 3:48 PM	Microsoft Excel Co...	697 KB
dim_delivery_agents		5/15/2025 3:48 PM	Microsoft Excel Co...	18 KB
dim_locations		5/15/2025 3:48 PM	Microsoft Excel Co...	7 KB
dim_menu_items		5/15/2025 3:48 PM	Microsoft Excel Co...	21 KB
dim_restaurants		5/15/2025 3:48 PM	Microsoft Excel Co...	10 KB

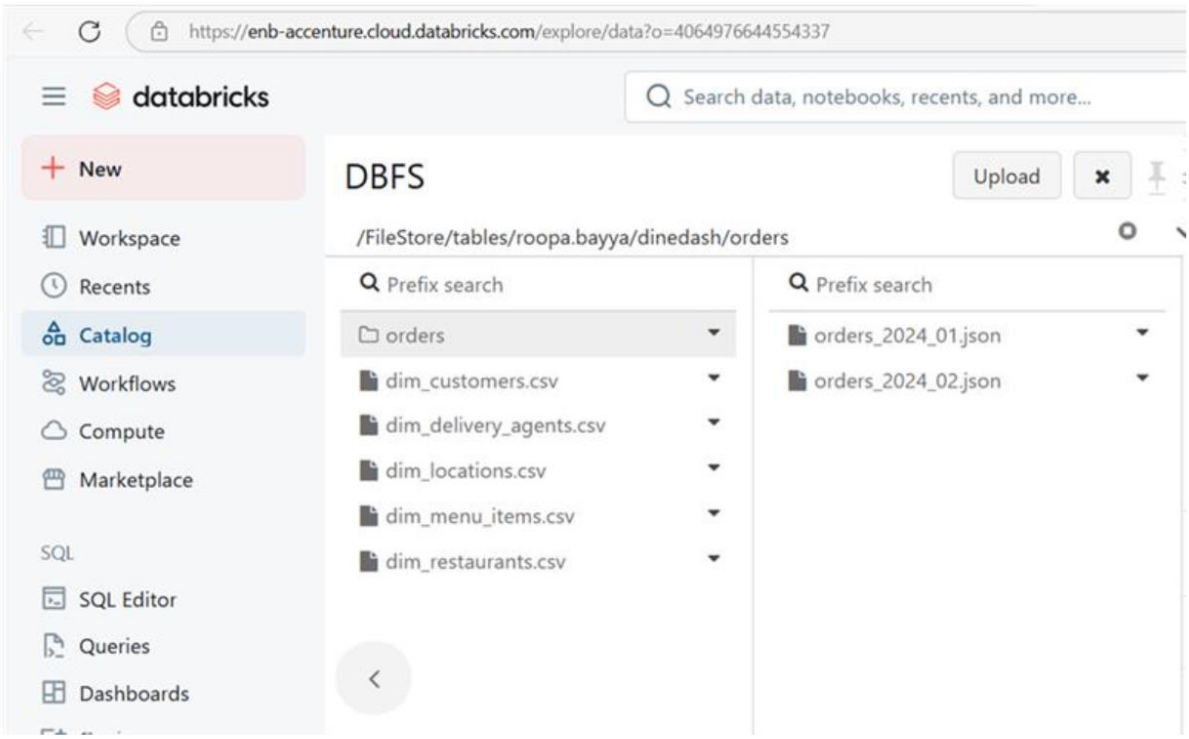
Data Ingestion

Manually Uploading datasets to DBFS

In **enb-accenture** databricks workspace

Create folder in DBFS as ***/FileStore/tables/<enterpriseid>/dinedash*** and upload the dataset as shown below.

For orders create a folder by the name **orders** withthin dinedash folder and upload only Jan and Feb orders datasets to begin with



Initial Data Exploration

Querying the files directly

Explore each of the dataset by querying against the DBFS path of each dataset files directly.Observe the sample records to understand about the dataset by displaying all the columns for 10 records.

Creating Tables

1. Create temporary views using the **read_files()** table-valued function by inferring schema from the data for each of the datasets for customers, restauarnts, locations, delivery_agents, menu_items as

customers_tmp_view,
restaurants_tmp_view,
locations_tmp_view,
delivery_agents_tmp_view,
menu_items_tmp_view

2.Create raw delta tables providing the manual shcem and capture the record **ingestion time** and **source file name** for each record.

Create tables for each dataset, namely

customers_bronze,
restaurants_bronze,
locations_bronze,
delivery_agents_bronze
menu_items_bronze from

Ensure that all tables have appropriate column names and data types

3.Create cleaned data delta tables

Create following tables from the corresponding bronze tables

customers_filtered

Which contains the records where customer_id **,** location_id, signup_date, dob, email, name are not missing

restaurants_ filtered

Which contains the records where restaurant_id, name, cuisines, location_id, rating, delivery_fee are not missing

locations_ filtered

Which contains the records where location_id, area, city, state, latitude, longitude are not missing

delivery_agents_ filtered

Which contains the records where agent_id,name,phone,rating are not missing

menu_items_filtered

Which contains the records where
item_id,restaurant_id,item_name,price,category are not missing

Incremental Data Loading

1. Create a table named **orders_bronze** without schema and incrementally load order details for each month of the year 2024, ingesting one month's data at a time**, using COPY INTO**

Ingest Janaury,Febraury month order details,verify the records count and then ingest March month order details, verify the records count

Problem Statements

Note: All numeric calculations should be rounded UP to two decimal places for floating-point numbers and one decimal place for discrete entities.
Price mentioned is in dollars($)

Order Data Analysis

Write Databricks SQL query to find

1. Total number of orders placed
2. Distribution of orders by payment method
3. Distribution of order status (e.g., delivered, preparing, cancelled)
4. Orders per restaurant (volume of business)
5. Orders per customer (loyalty insight)

6. Orders handled per delivery agent
7. Top 10 orders by total amount
8. Average tip amount per payment method
9. Percentage of orders with tip > 10% of total_amount
10. Percentage of successful vs cancelled vs failed orders.
11. Top customers (by number of orders) and top restaurants (by number of orders).
12. Item-Level Insights (explode items_ordered)
    - Find most frequently ordered items
    - Find average number of items per order
    - Find item revenue contribution per restaurant

Orders Data and Dimension Datasets Analysis

1. Find the preferred restaurant names for each customer
2. Find the top 10 areas receiving orders in each month
3. Find all successfully delivered orders that were paid using a card, to analyze payment preferences and delivery success rates
4. Which cuisines generate the highest average order value?
   Hint: Join orders ,restaurants, group by cuisine, and calculate average total_amount.
5. Which delivery agents have delivered the most high-value orders (above $50)?
   Hint: Filter orders for total_amount > 50, join with agent, group by agent_id, count orders.
6. Which cities (area_name) are generating the most revenue for Italian restaurants?
   Hint: Join orders , restaurants , locations, filter cuisine = 'Italian', group by area_name, sum total_amount
7. What is the tip-to-total ratio by cuisine type?
   Hint: Join orders , restaurants, group by cuisine, calculate average tip / total_amount.

8. Which menu items are most frequently ordered, and from which restaurants?

Hint: Explode items_ordered in orders, join with menu_items, group by item_name, restaurant_id, count.

9. Which agents have delivered the most diverse cuisines?

Hint: Join orders , restaurants, group by agent_id, count distinct cuisine.

10. What is the average customer lifetime value by signup year?

Hint: Join orders , customers, extract year from signup_date, group by year, sum total_amount per customer, then average total_amount per customer, then average

11. Which restaurants have the highest order frequency per location?

Hint: Join orders , restaurants , locations, group by restaurant_id, location_id, count orders.

12. Which locations are prone to order cancellations?

Hint: Filter orders with order_status = 'cancelled', join locations using delivery_location_id, group by area_name, count.

13. Find top 10 customers who have ordered from the widest variety of restaurants.

Hint: Join orders, group by customer_id, count distinct restaurant_id, sort and pick top 10.

14. Identify restaurants with frequent repeat customers.

Hint: Join orders, group by restaurant_id, customer_id, count, filter where count > 1.

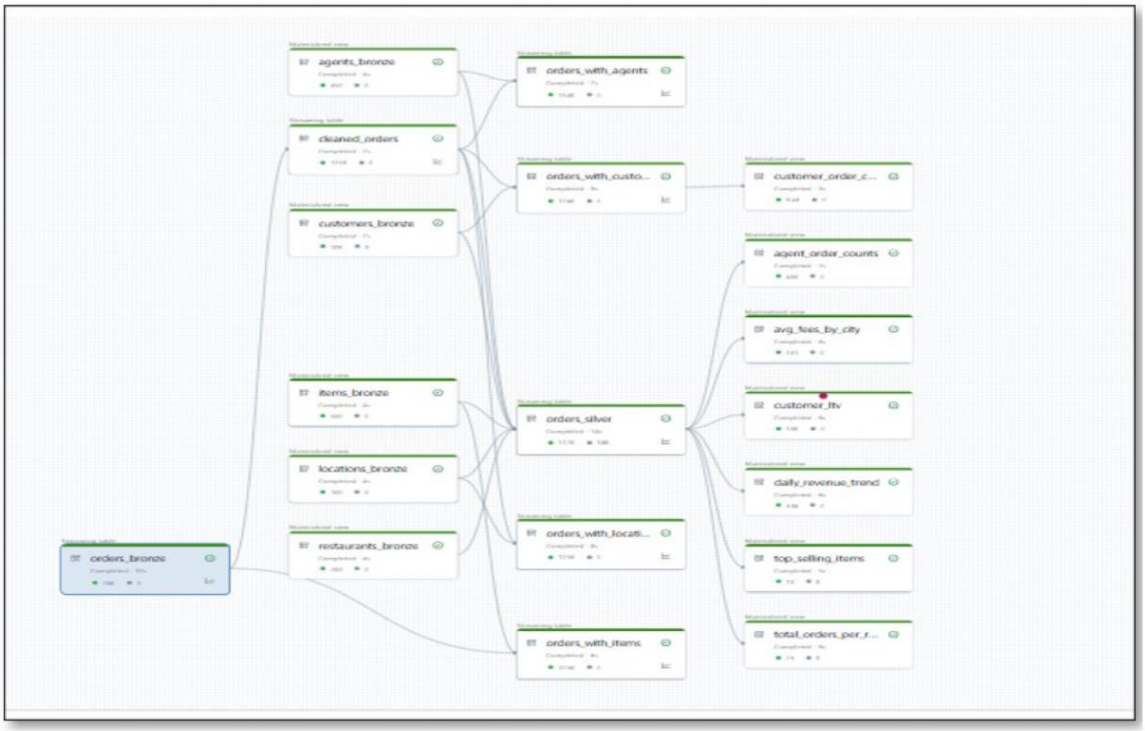
15. Which payment methods are most preferred for high-value orders across cuisines?

Hint: Filter orders with total_amount > 40, join restaurants, group by cuisine, payment_method, count.

Guided Activity

Delta Live Tables

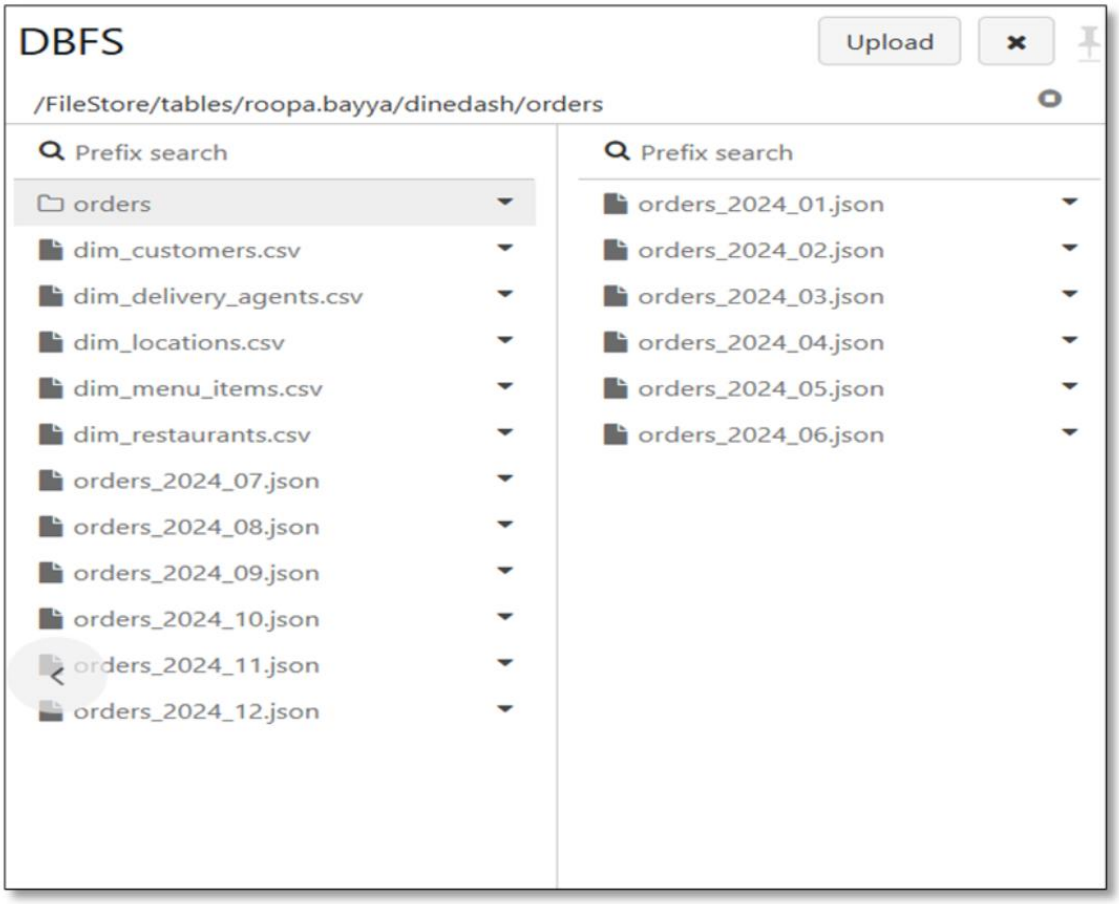
Final Executed Pipeline View: This is what we build !



NOTE: To begin with, have orders details till June month in orders folder '*dbfs:/FileStore/tables/roopa.bayya/dinedash/orders*'

Execute below in some notebook to move order details from June month onwards to other folder

```
%python
for month in range(7, 13):  # 07 to 12
    src = f"dbfs:/FileStore/tables/roopa.bayya/dinedash/orders/orders_2024_{month:02}.json"
    dst = f"dbfs:/FileStore/tables/roopa.bayya/dinedash/orders_2024_{month:02}.json"
    dbutils.fs.mv(src, dst)
```


DBFS
Upload
/FileStore/tables/roopa.bayya/dinedash/orders
Prefix search
orders
dim_customers.csv
dim_delivery_agents.csv
dim_locations.csv
dim_menu_items.csv
dim_restaurants.csv
orders_2024_07.json
orders_2024_08.json
orders_2024_09.json
orders_2024_10.json
orders_2024_11.json
orders_2024_12.json
Prefix search
orders_2024_01.json
orders_2024_02.json
orders_2024_03.json
orders_2024_04.json
orders_2024_05.json
orders_2024_06.json

Steps to Cretae DLT pipeline:

Follow through the steps as depicted in screenshots below

Step 1: Create Script

Create a notebook by the name Dinedash_DLT_PL and write the below code into it

```
-- BRONZE LAYER: Ingest Raw Data

CREATE OR REFRESH STREAMING TABLE orders_bronze AS
SELECT * FROM STREAM READ_FILES(
  'dbfs:/FileStore/tables/roopa.bayya/dinedash/orders',
  format => 'json'
);

CREATE LIVE TABLE restaurants_bronze AS
SELECT * FROM READ_FILES(
  'dbfs:/FileStore/tables/roopa.bayya/dinedash/dim_restaurants.csv',
  format => 'csv',
  inferSchema => true,
  header => true
);
```

```
CREATE LIVE TABLE items_bronze AS
SELECT * FROM READ_FILES(
  'dbfs:/FileStore/tables/roopa.bayya/dinedash/dim_menu_items.csv',
  format => 'csv',
  inferSchema => true,
  header => true
);


CREATE LIVE TABLE customers_bronze AS
SELECT * FROM READ_FILES(
  'dbfs:/FileStore/tables/roopa.bayya/dinedash/dim_customers.csv',
  format => 'csv',
  inferSchema => true,
  header => true
);

CREATE LIVE TABLE agents_bronze AS
SELECT * FROM READ_FILES(

'dbfs:/FileStore/tables/roopa.bayya/dinedash/dim_delivery_agents.csv',
  format => 'csv',
  inferSchema => true,
  header => true
);


CREATE LIVE TABLE locations_bronze AS
SELECT * FROM READ_FILES(
  'dbfs:/FileStore/tables/roopa.bayya/dinedash/dim_locations.csv',
  format => 'csv',
  inferSchema => true,
  header => true
);
```

```
-- SILVER LAYER: Clean + Normalize
CREATE OR REFRESH STREAMING LIVE TABLE cleaned_orders (
  CONSTRAINT valid_order_id EXPECT (order_id IS NOT NULL) ON
VIOLATION DROP ROW,
  CONSTRAINT valid_customer EXPECT (customer_id IS NOT NULL) ON
VIOLATION DROP ROW,
  CONSTRAINT valid_items EXPECT (size(items_ordered) > 0) ON
VIOLATION DROP ROW
)
AS
SELECT
  order_id,
  timestamp as order_ts,
  customer_id,
  restaurant_id,
  agent_id,
  delivery_location_id,
  items_ordered,
  explode(items_ordered) AS item,
  total_amount,
  tip,
  payment_method,
  order_status
FROM STREAM(Live.orders_bronze);
```

```
CREATE OR REFRESH STREAMING TABLE orders_silver
(
  CONSTRAINT valid_restaurant_name EXPECT (restaurant_name IS NOT NULL) ON VIOLATION DROP ROW,
  CONSTRAINT valid_agent EXPECT (agent_name IS NOT NULL) ON VIOLATION DROP ROW,
  CONSTRAINT valid_item EXPECT (item_id IS NOT NULL) ON VIOLATION DROP ROW
)
AS
SELECT
  o.order_id,
  o.order_ts,
  o.customer_id,
  c.name AS customer_name,
  c.email,
  o.restaurant_id,
  r.name AS restaurant_name,
  r.cuisines,
  r.rating AS restaurant_rating,
  o.agent_id,
  a.name AS agent_name,
  a.rating AS agent_rating,
```

```
  o.delivery_location_id,
  l.city,
  o.item.item_id AS item_id,
  i.item_name,
  i.category,
  o.item.price AS item_price,
  o.item.quantity AS item_quantity,
  o.total_amount,
  o.tip,
  o.payment_method,
  o.order_status
FROM STREAM(Live.cleaned_orders) o
LEFT JOIN Live.customers_bronze c ON o.customer_id = c.customer_id
LEFT JOIN Live.restaurants_bronze r ON o.restaurant_id =
r.restaurant_id
LEFT JOIN Live.agents_bronze a ON o.agent_id = a.agent_id
LEFT JOIN Live.items_bronze i ON o.item.item_id = i.item_id
LEFT JOIN Live.locations_bronze l ON o.delivery_location_id =
l.location_id;


  CREATE OR REFRESH STREAMING TABLE orders_with_agents AS
  SELECT
    o.order_id,
    o.order_ts,
    o.agent_id,
    a.name AS agent_name,
    a.rating AS agent_rating,
    o.tip
  FROM STREAM(Live.cleaned_orders) o
  LEFT JOIN agents_bronze a
  ON o.agent_id = a.agent_id;
```

```
CREATE OR REFRESH STREAMING TABLE orders_with_items AS
SELECT
  o.order_id,
  o.order_ts,
  o.restaurant_id,
  exploded.item_id,
  i.item_name,
  i.category,
  exploded.price,
  exploded.quantity
FROM  (
  SELECT order_id, timestamp as order_ts, restaurant_id,
explode(items_ordered) AS exploded
  FROM STREAM(Live.orders_bronze)
) o
LEFT JOIN items_bronze i
ON o.exploded.item_id = i.item_id;


CREATE OR REFRESH STREAMING TABLE orders_with_locations AS
SELECT
  o.order_id,
  o.order_ts,
  o.delivery_location_id,
  l.city,
  l.state,
  l.area
FROM STREAM(Live.cleaned_orders) o
LEFT JOIN locations_bronze l
ON o.delivery_location_id = l.location_id;
```

```
-- GOLD LAYER: Insightful Metrics

-- 1. Total orders per restaurant
CREATE LIVE TABLE total_orders_per_restaurant AS
SELECT restaurant_name, COUNT(DISTINCT order_id) AS total_orders
FROM orders_silver
GROUP BY restaurant_name;

-- 2. Top-selling items
CREATE LIVE TABLE top_selling_items AS
SELECT item_name, SUM(item_quantity) AS total_qty_sold
FROM orders_silver
GROUP BY item_name
ORDER BY total_qty_sold DESC
LIMIT 10;
```

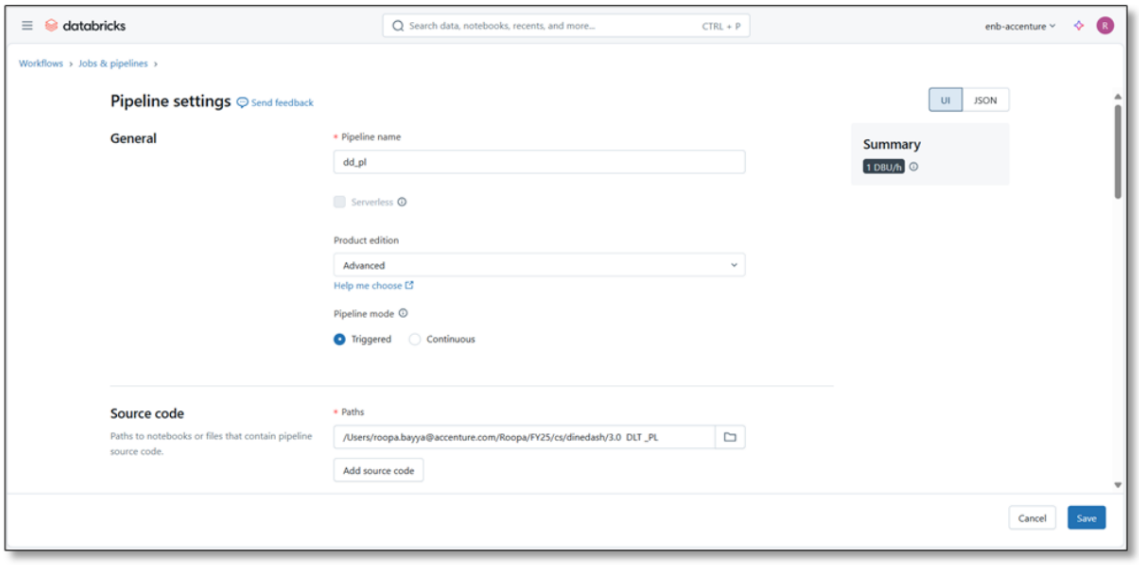
```
-- 3. Avg delivery fee and tip by city
CREATE LIVE TABLE avg_fees_by_city AS
SELECT city, AVG(restaurant_rating) AS avg_restaurant_rating, AVG(tip)
AS avg_tip
FROM orders_silver
GROUP BY city;

-- 4. Agent performance
CREATE LIVE TABLE agent_order_counts AS
SELECT agent_name, COUNT(DISTINCT order_id) AS orders_handled,
AVG(agent_rating) AS avg_rating
FROM orders_silver
GROUP BY agent_name
ORDER BY orders_handled DESC;

-- 5. Revenue trend
CREATE LIVE TABLE daily_revenue_trend AS
SELECT DATE(order_ts) AS order_date, SUM(total_amount) AS
total_revenue
FROM Live.orders_silver
GROUP BY order_date
ORDER BY order_date;
```

```
-- 6. Customer lifetime value
CREATE LIVE TABLE customer_ltv AS
SELECT customer_id, customer_name, SUM(total_amount) AS
lifetime_spend
FROM orders_silver
GROUP BY customer_id, customer_name
ORDER BY lifetime_spend DESC;
CREATE OR REFRESH STREAMING TABLE orders_with_customers AS
SELECT
  o.order_id,
  o.customer_id,
  c.name AS customer_name,
  c.location_id AS customer_location_id,
  o.total_amount,
  o.tip,
  o.payment_method,
  o.order_status
FROM STREAM(Live.cleaned_orders) o
LEFT JOIN Live.customers_bronze c
ON o.customer_id = c.customer_id;
```

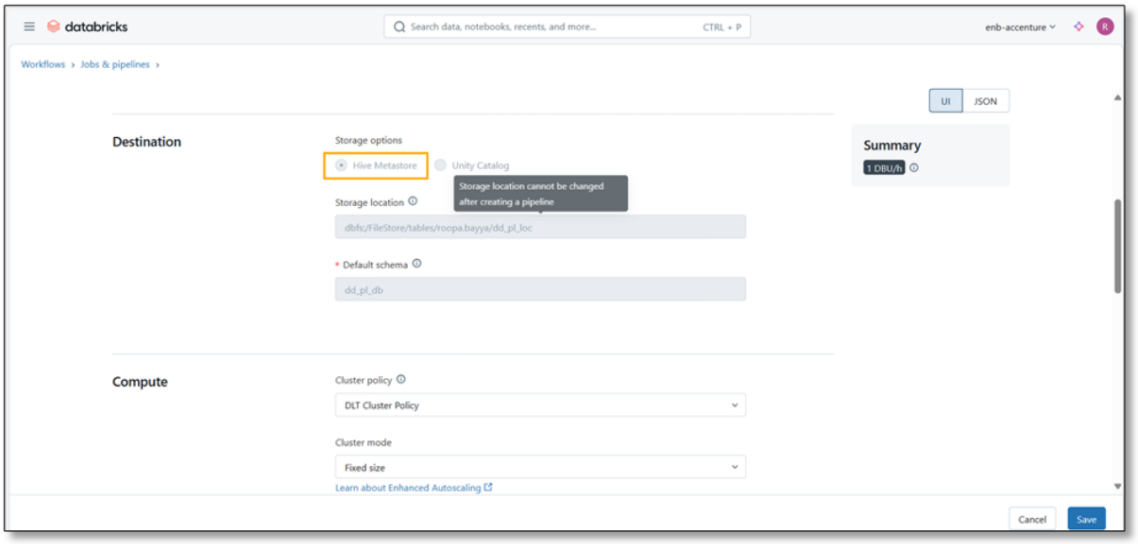
```
CREATE LIVE TABLE customer_order_counts AS
SELECT customer_name, COUNT(*) AS total_orders
FROM orders_with_customers
GROUP BY customer_name;
```

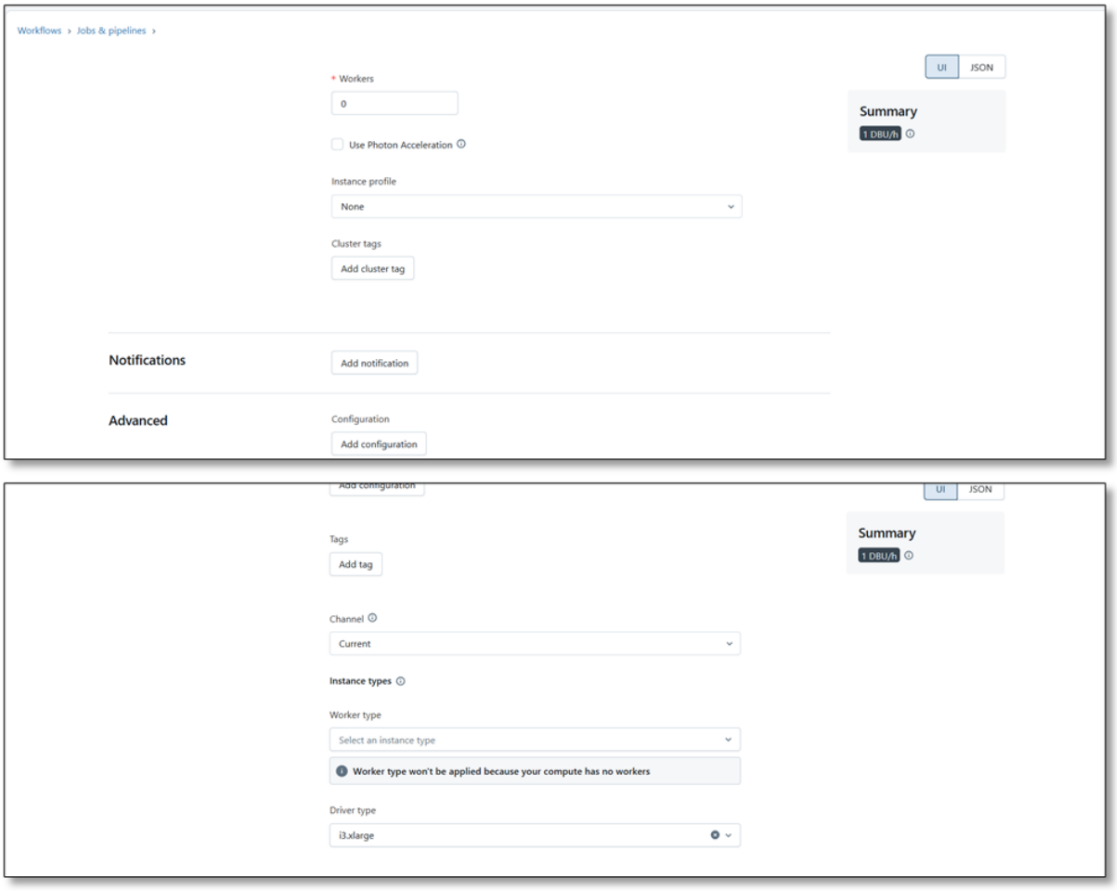
Step 2: Create Pipeline

Goto Workflows menu in Databricks Workspace, select Jobs and Pipelines tab and fill the details as shown below



Ensure that Storage Location and Default schema are unique for you

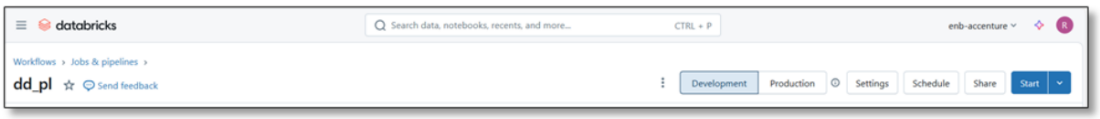




Click on **Create**

Step 3: Start the pipeline manually

Click on **Start**



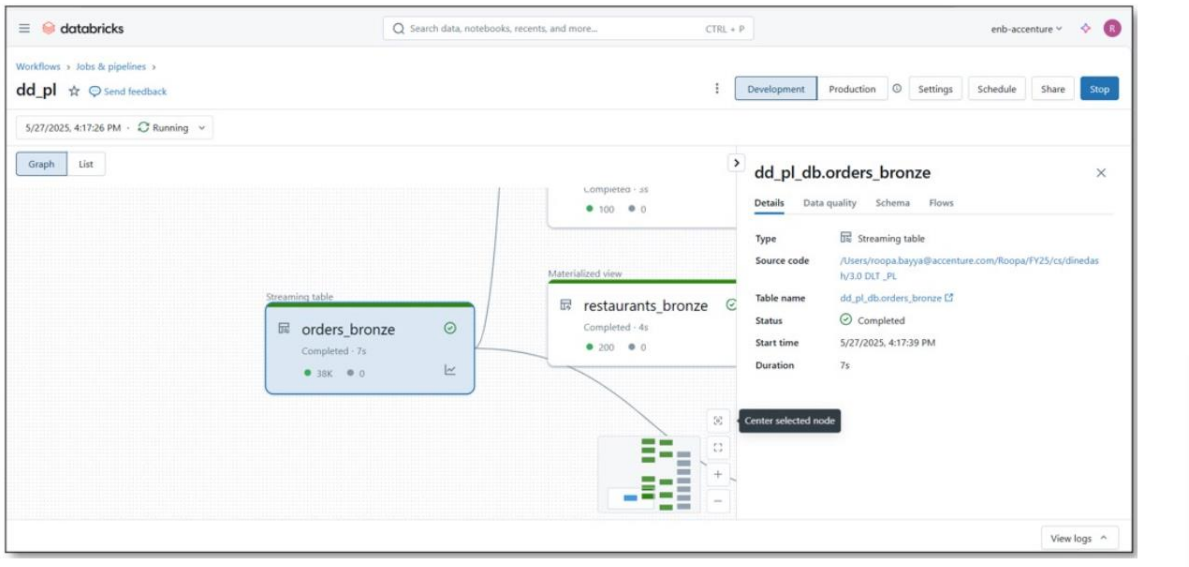
Initially it will take 5-10 for the pipeline to run, as the cluster is being set up.

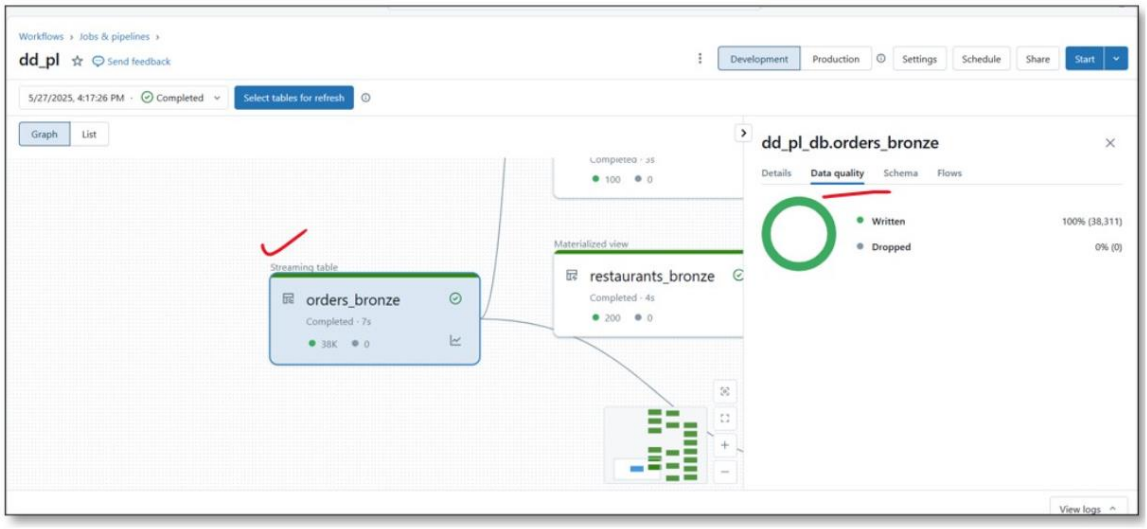
Once the pipeline is executed, check through the details for each of the table. Below you see the Zoomed out screenshots for the pipeline

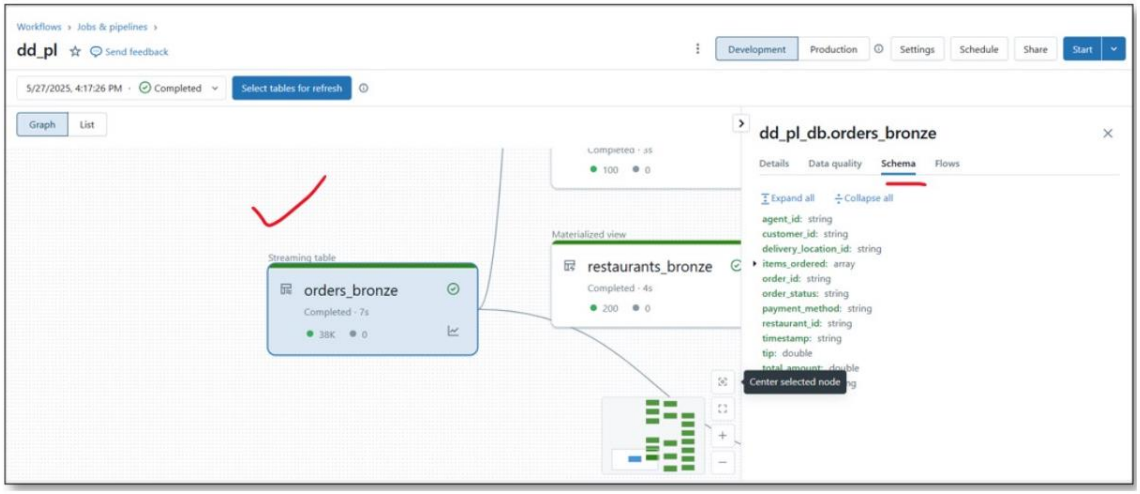

Materialized view
items_bronze
Completed · 4s
500 0
Materialized view
locations_bronze
Completed · 3s
100 0
Materialized view
restaurants_bronze
Completed · 4s
200 0
Streaming table
orders_bronze
Completed · 7s
38K 0
Streaming table
orders_silver
Completed · 11s
59K 19K
Streaming table
orders_with_locati...
Completed · 7s
63K 0
Materialized view
avg_fees_by_city
Completed · 4s
101 0
Materialized view
customer_ltv
Completed · 5s
9.4K 0
Materialized view
daily_revenue_trend
Completed · 5s
168 0
Materialized view
top_selling_items
Completed · 4s
10 0


items_bronze
Completed · 4s
500 0
Materialized view
locations_bronze
Completed · 3s
100 0
Materialized view
restaurants_bronze
Completed · 4s
200 0
Streaming table
orders_bronze
Completed · 7s
38K 0
Streaming table
orders_silver
Completed · 11s
59K 19K
Streaming table
orders_with_locati...
Completed · 7s
63K 0
Streaming table
orders_with_items
Completed · 6s
63K 0
Materialized view
customer_ltv
Completed · 5s
9.4K 0
Materialized view
daily_revenue_trend
Completed · 5s
168 0
Materialized view
top_selling_items
Completed · 4s
10 0
Materialized view
total_orders_per_r...
Completed · 4s
25 0

Select the required tables and observe through details related to that table.Details for orders_bronze table is highlighted for you.



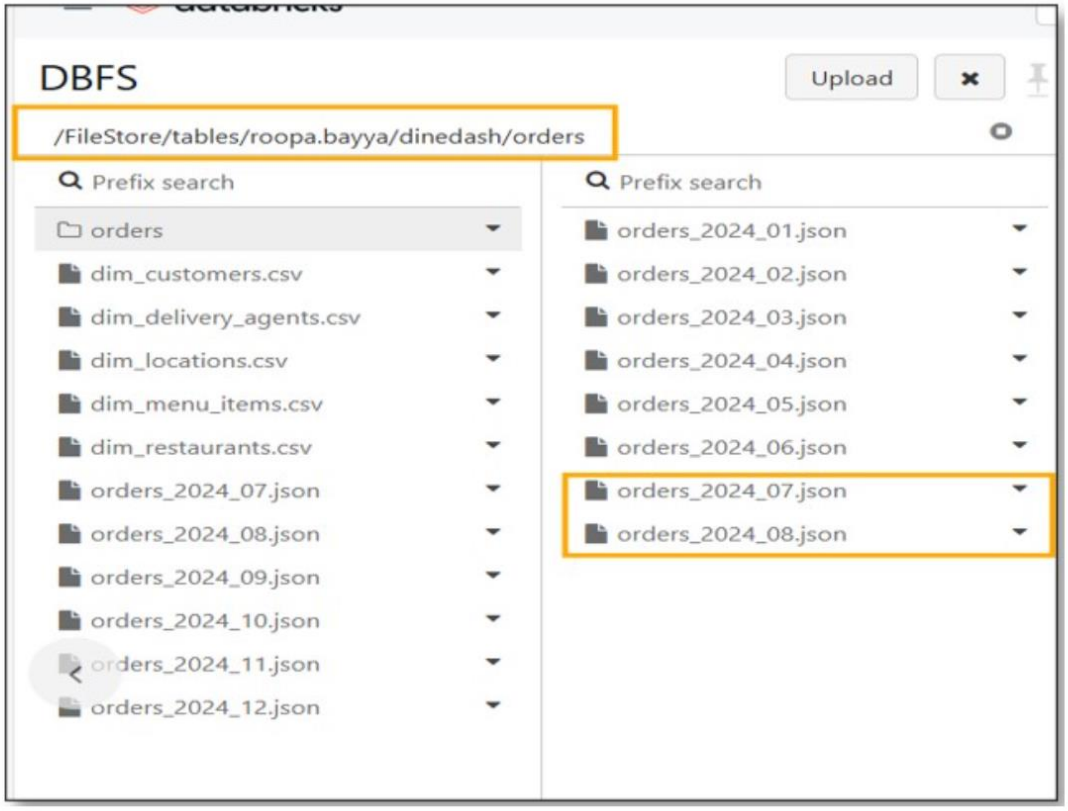




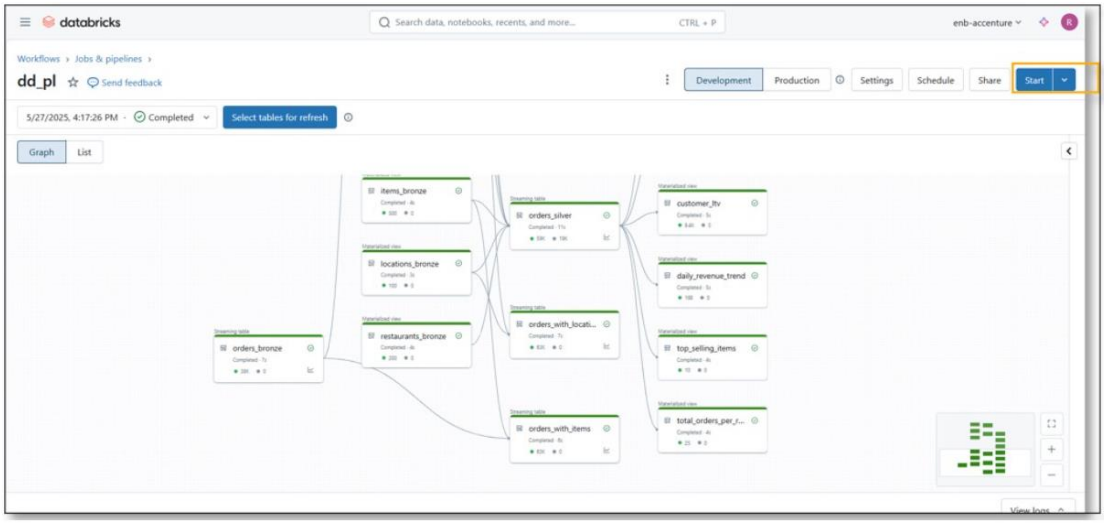
Step 4: Add new files to orders folder

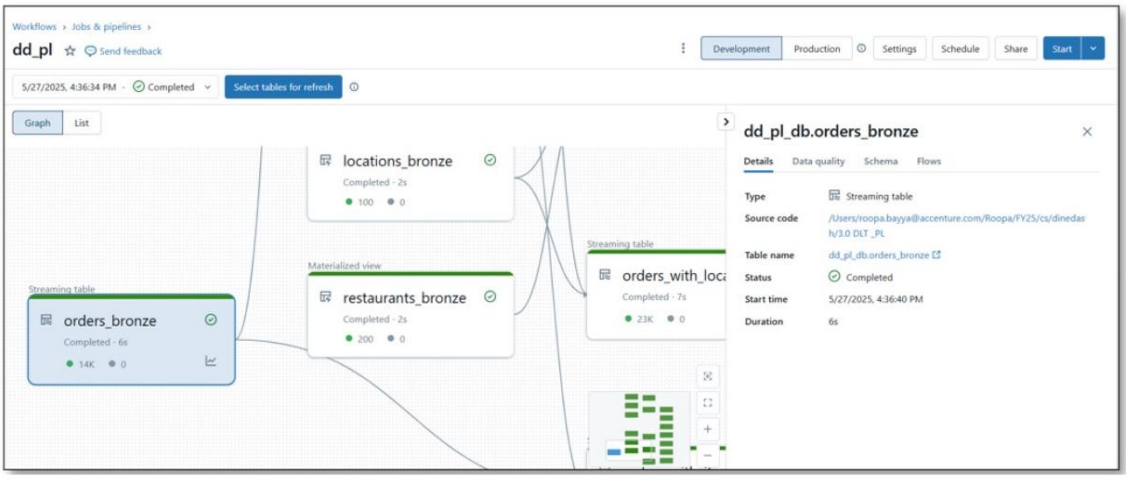
Now, add orders details for July and Aug month to the orders folder by executing below code in some notebook

```
%python
for month in range(7, 9):
    src = f"dbfs:/FileStore/tables/roopa.bayya/dinedash
/orders_2024_{month:02}.json"
    dst = f"dbfs:/FileStore/tables/roopa.bayya/dinedash/orders
/orders_2024_{month:02}.json"
    dbutils.fs.mv(src, dst)
```

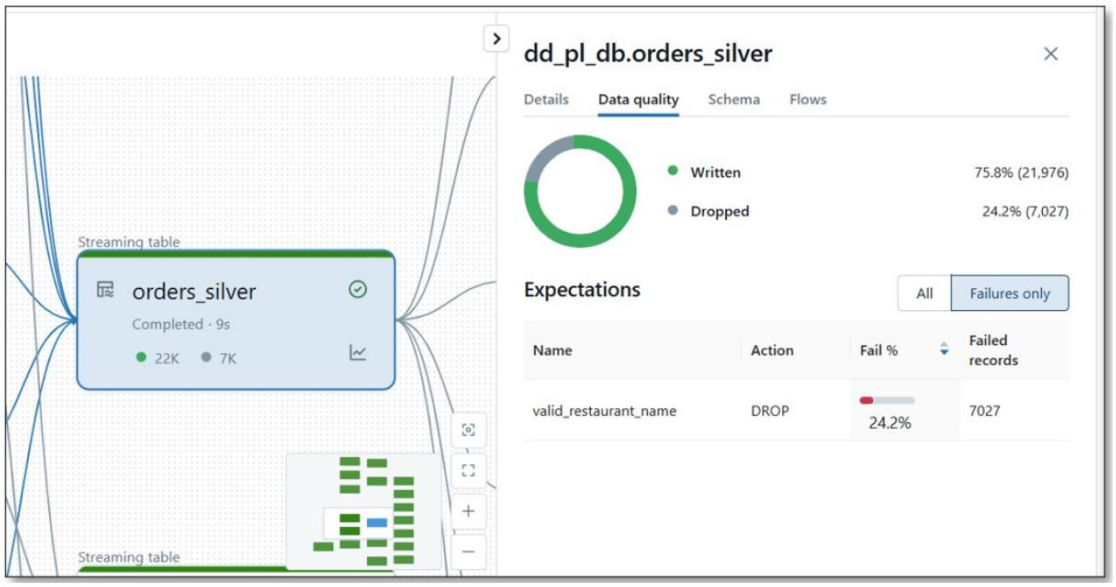


Step 5: Click on Start to execute the pipeline to feed data incrementally





Observe for any dropped records and the reason for the same



Step 6: Add remaining files onto orders folder

Step 7: Click on Start to execute the pipeline to feed data incrementally

Workflows

You will create a workflow to run the pipeline and publish some important insights onto Dashboard.

For this you creata a Job in Databricks Workspace.

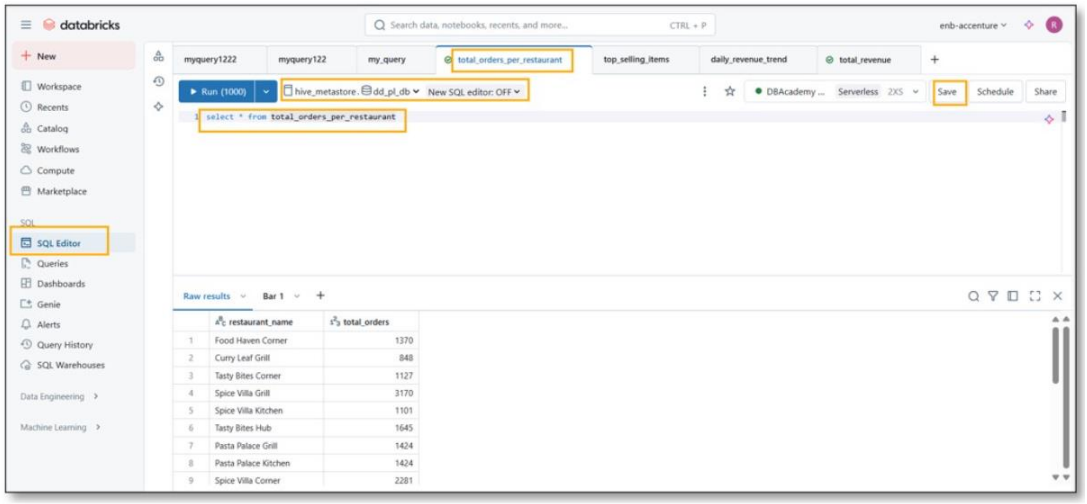
The job you create will have following flow:

Task 1 : DLT Pipeline (which you have created earlier) exection

Task 2 : Creating Dashboard to display total_orders_per-restaurant, daily_revenue_trends, top_selling_items, total_revenue

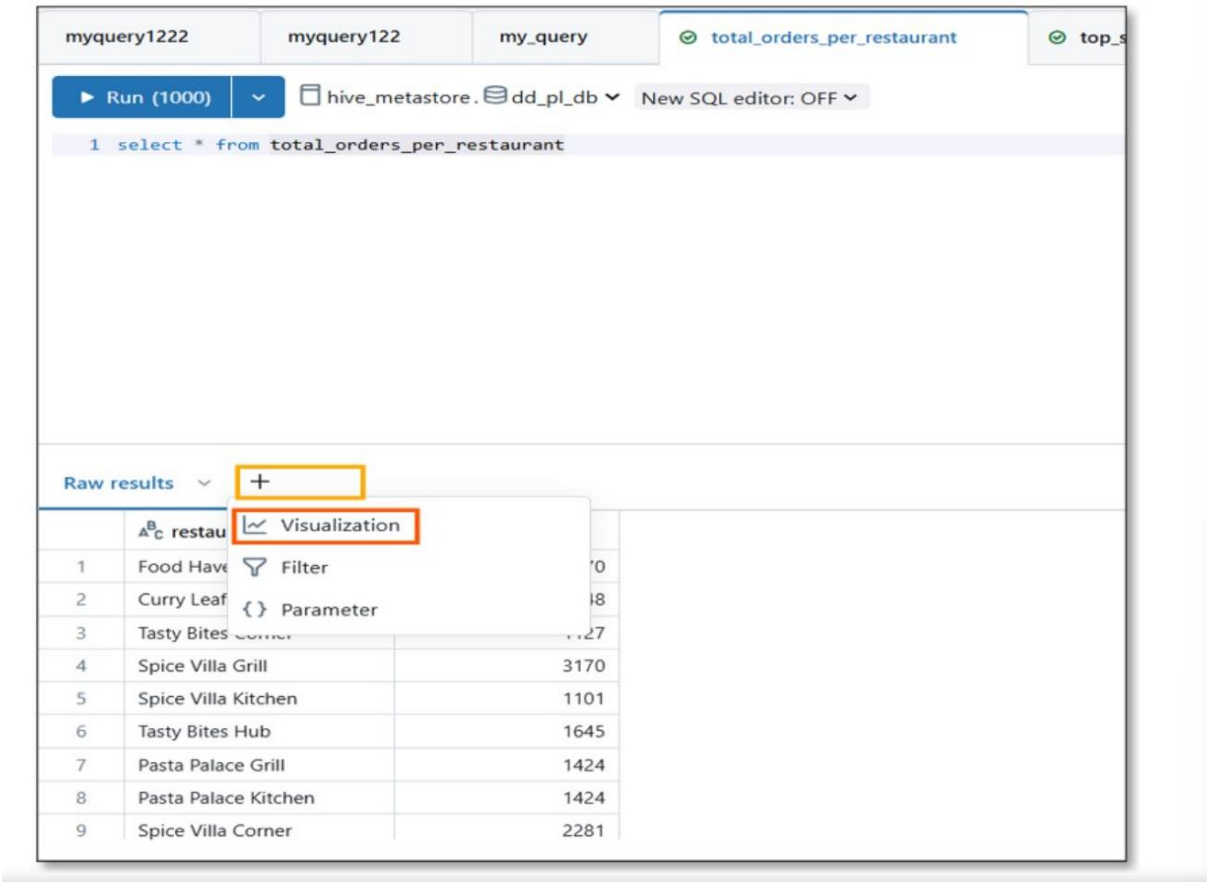
Dashboards are fed from gold layer table query results and we add visualization on top of it

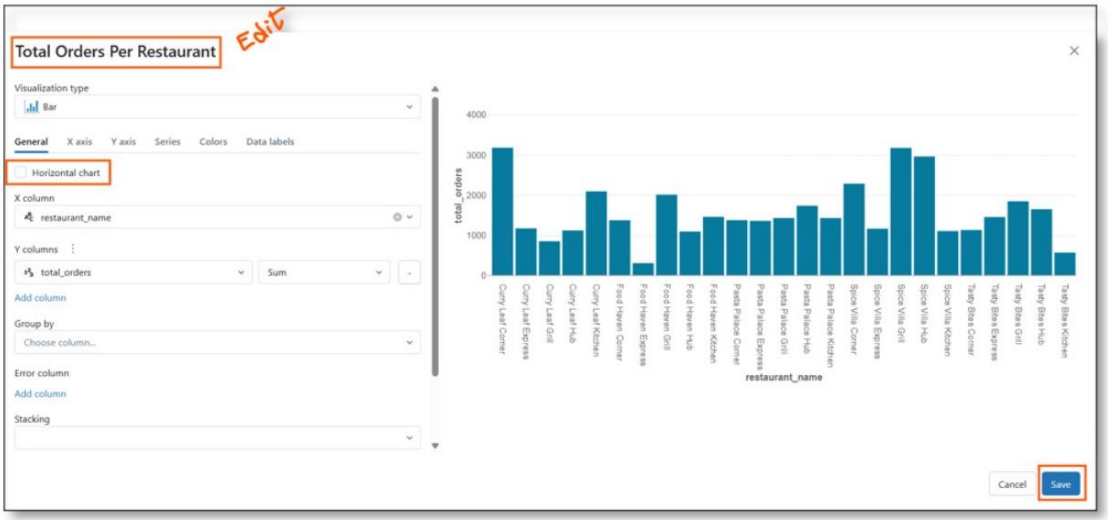
So, first create the queries as shown below and save them



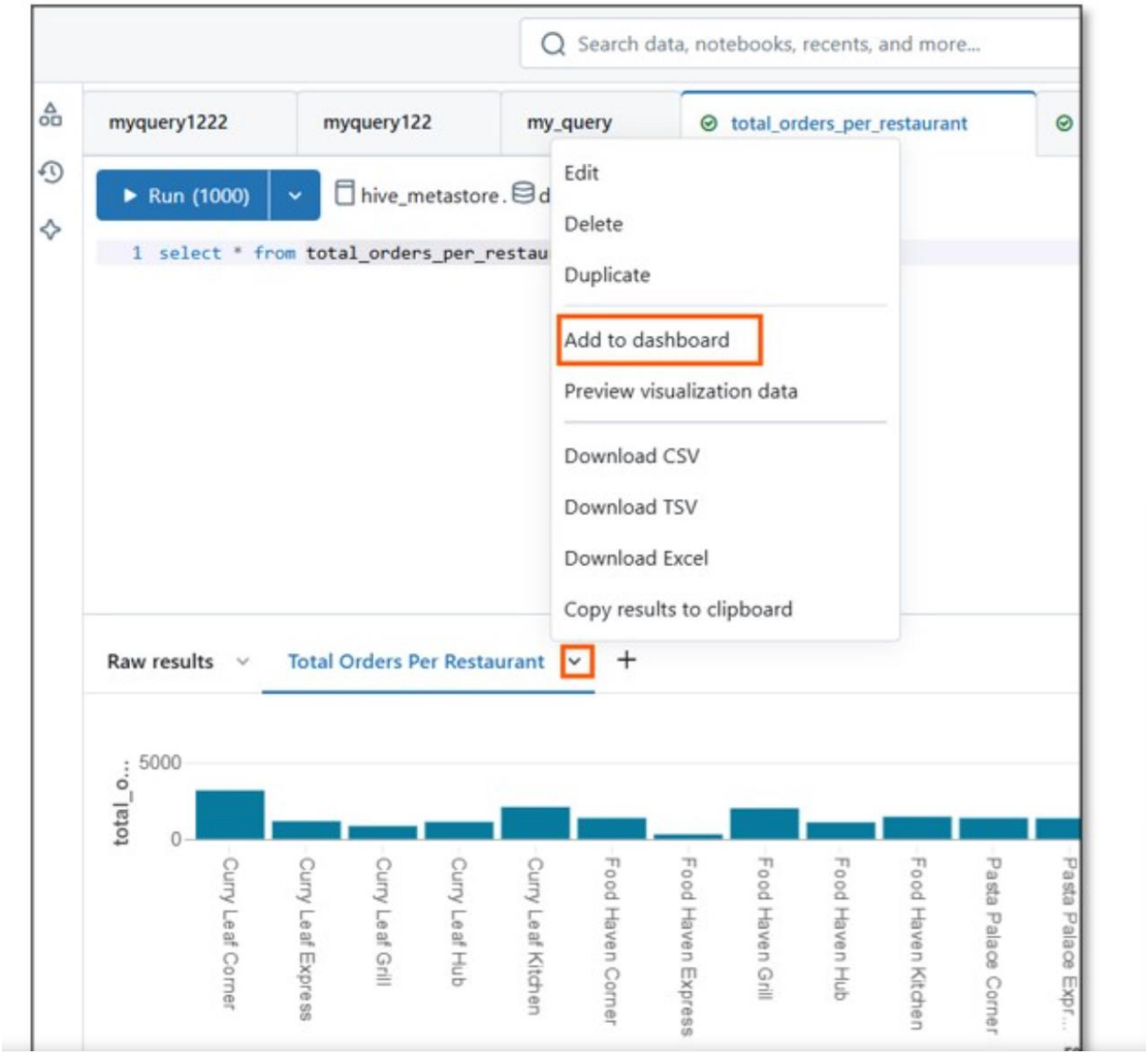
After saving , execute the query.

Choose + symbol next to Raw Results and create the visualization

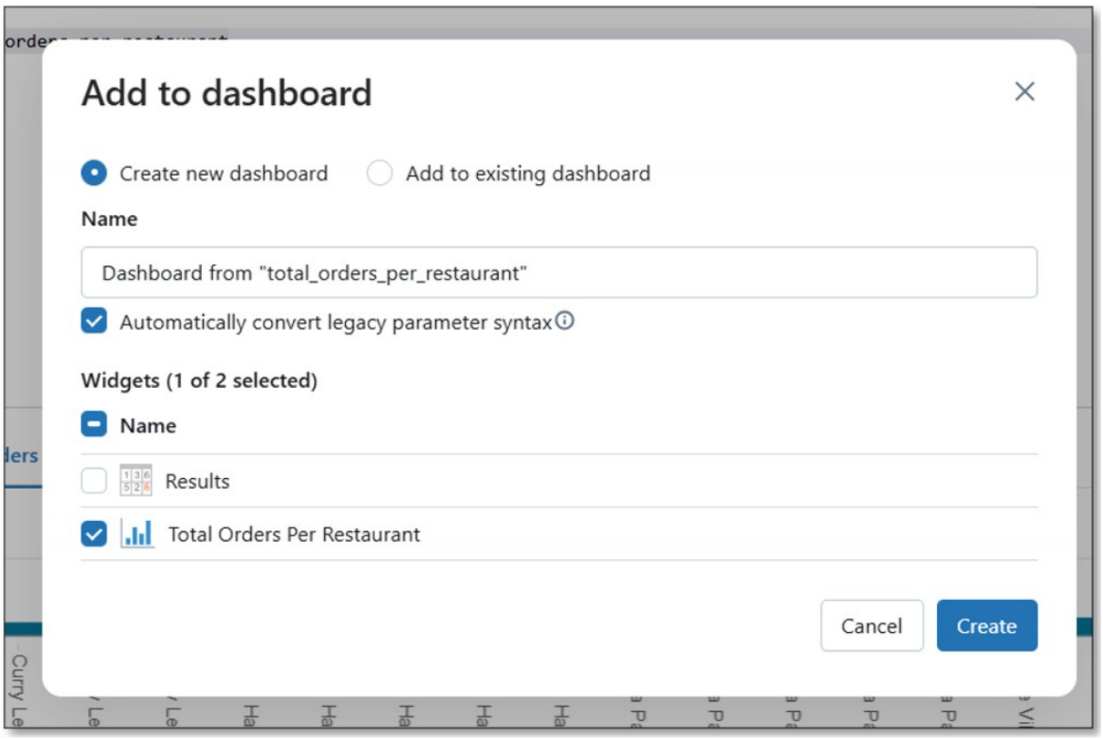


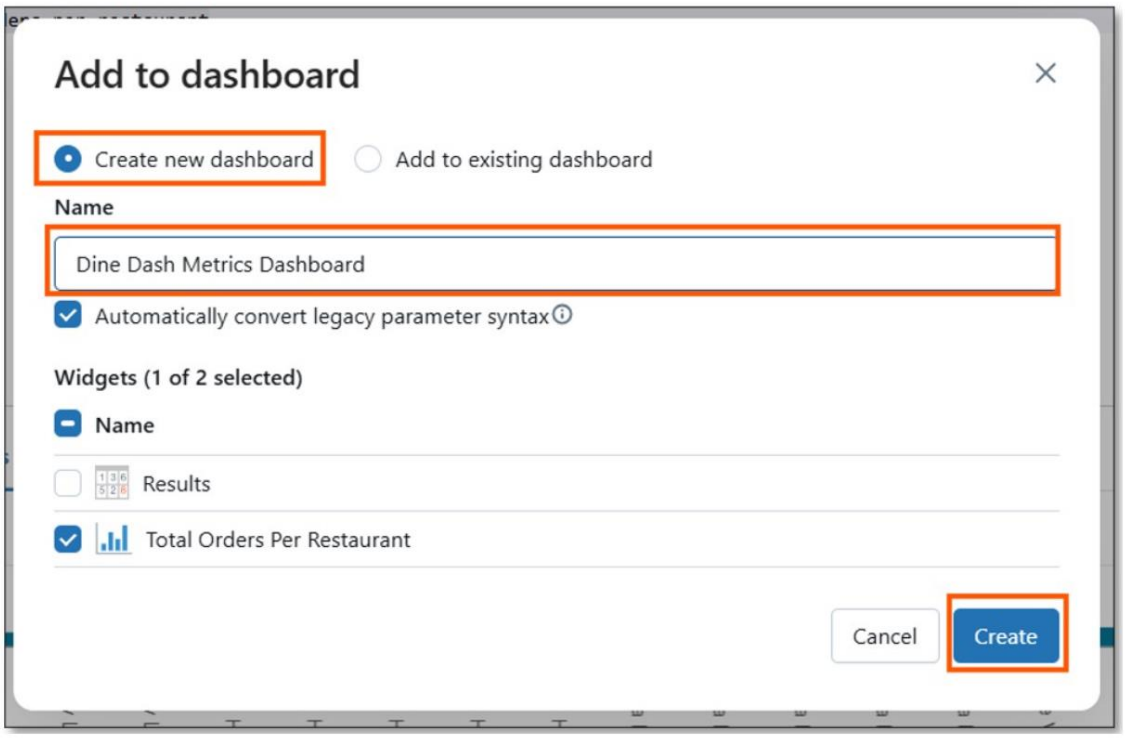


Click on downward arrow symbol on the visualization that you created and add it to the dashboard

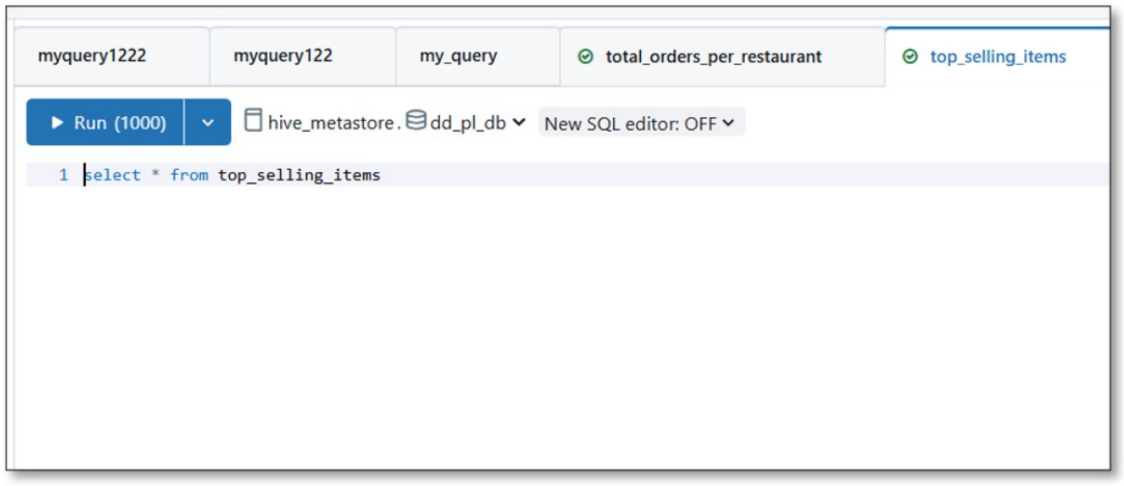

Search data, notebooks, recents, and more...
myquery1222
myquery122
my_query
total_orders_per_restaurant
Run (1000)
hive_metastore.
1 select * from total_orders_per_restau
Edit
Delete
Duplicate
Add to dashboard
Preview visualization data
Download CSV
Download TSV
Download Excel
Copy results to clipboard
Raw results
Total Orders Per Restaurant
total_o...
5000
0
Curry Leaf Corner
Curry Leaf Express
Curry Leaf Grill
Curry Leaf Hub
Curry Leaf Kitchen
Food Haven Corner
Food Haven Express
Food Haven Grill
Food Haven Hub
Food Haven Kitchen
Pasta Palace Corner

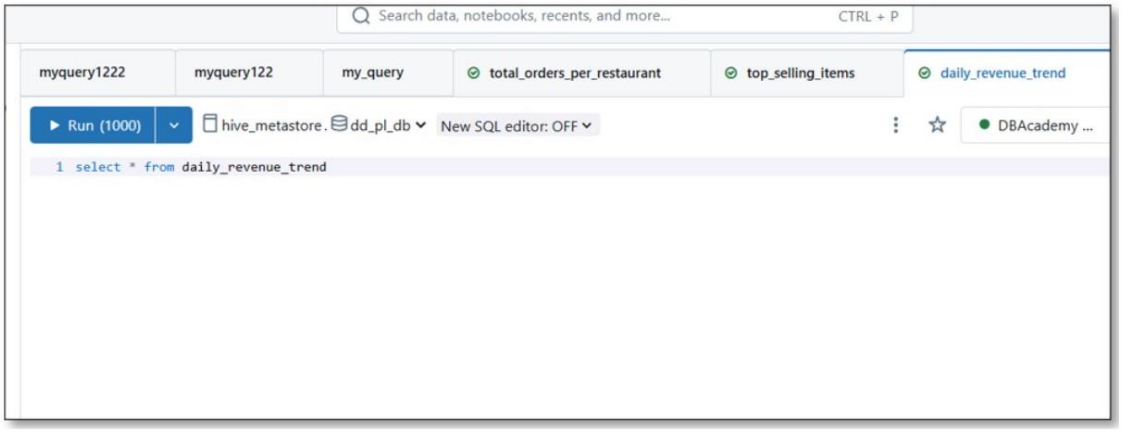
Create new Dashboard, give a name

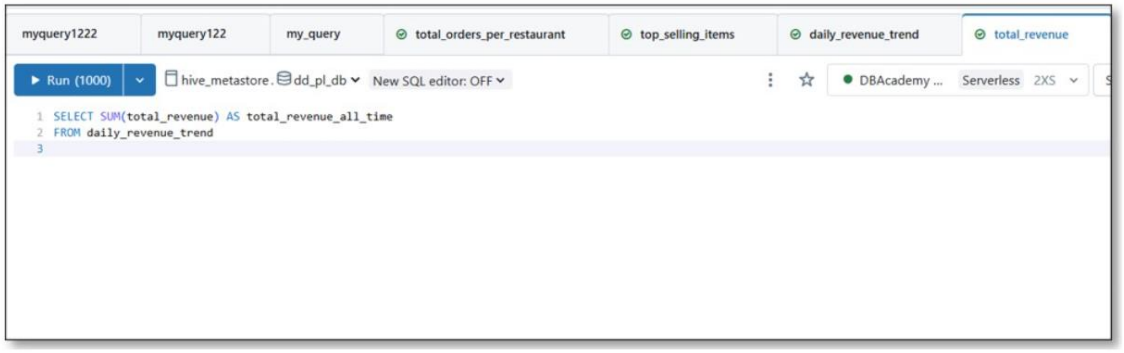




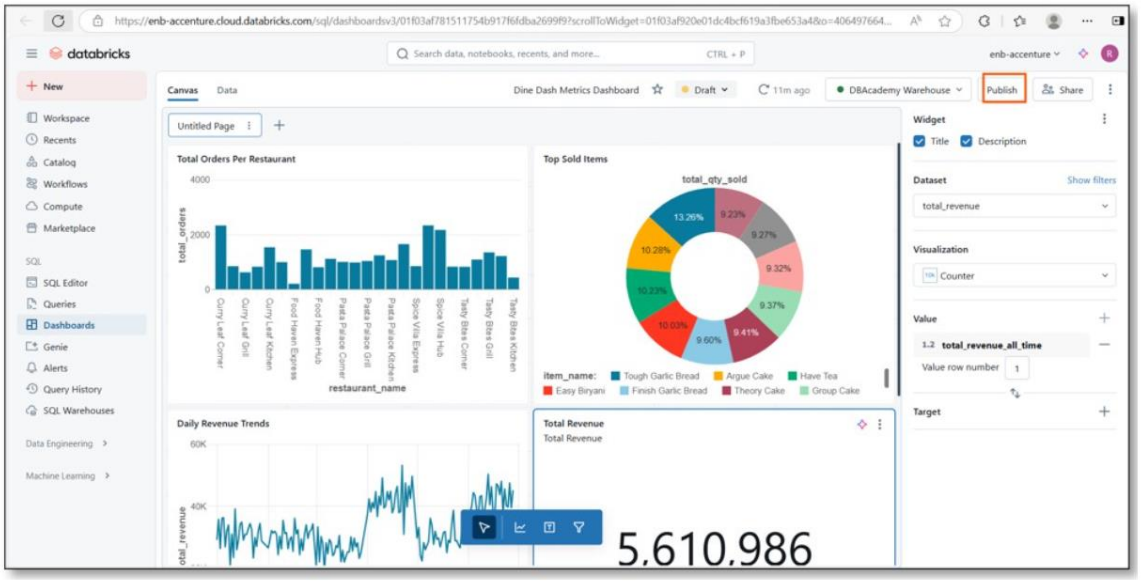
Repeat same way for below queries, choose appropriate visualizations



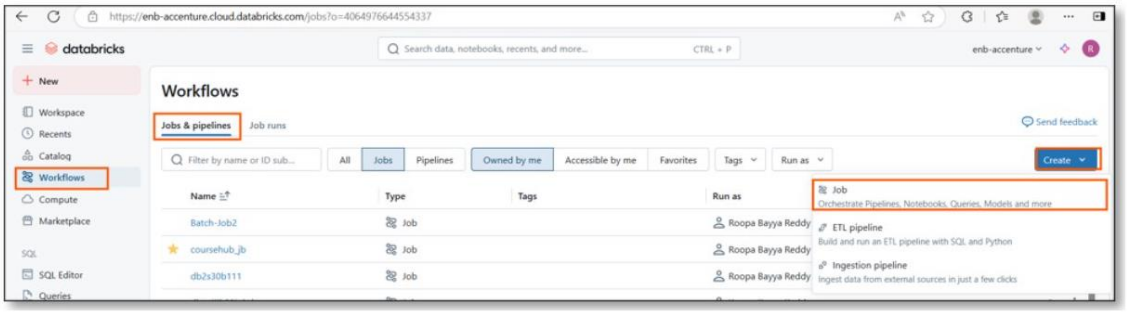


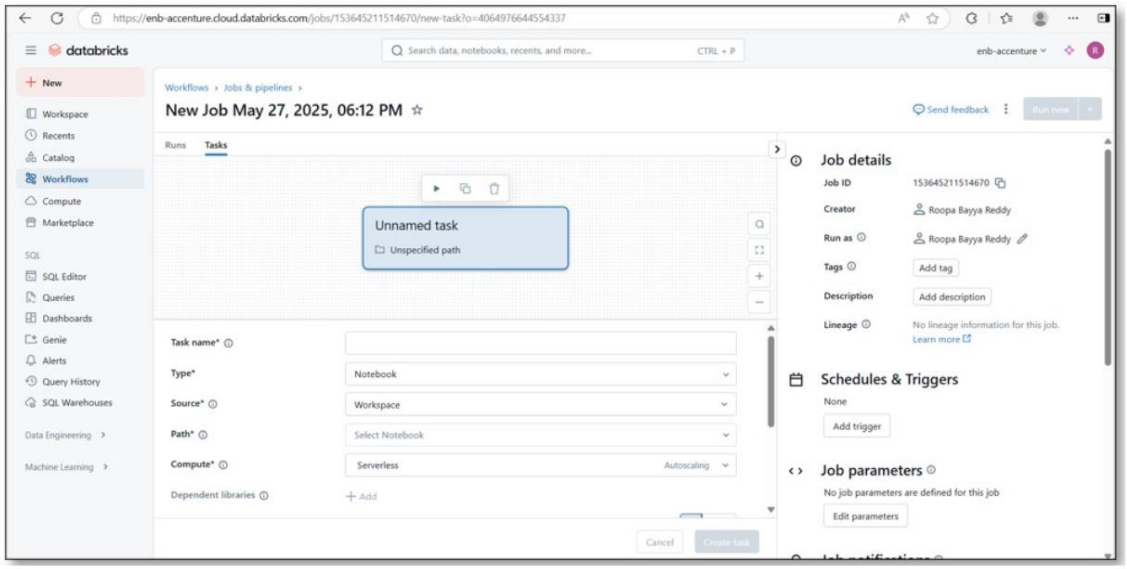


Once done, Goto DashBoards menu in the workspace and **publish the created dashboard** selecting default options

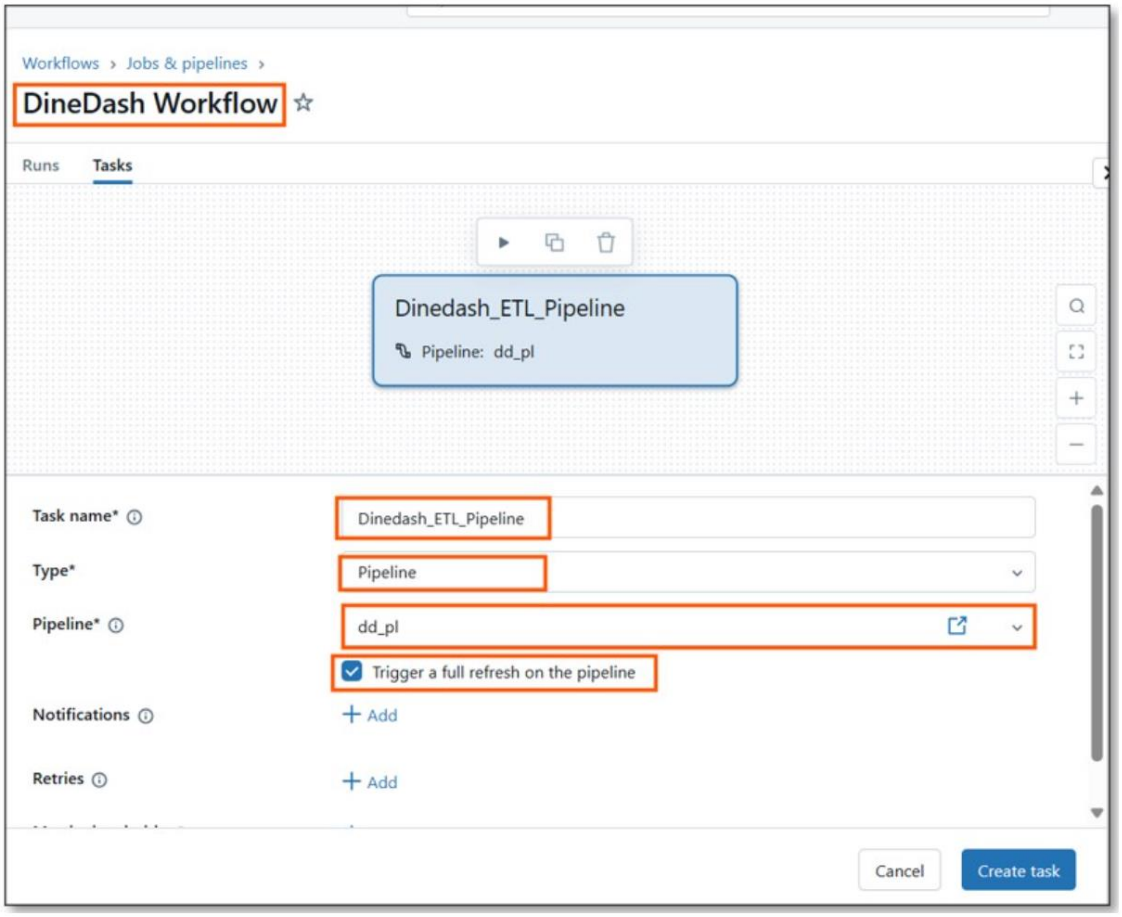


Creating Workflow/Job

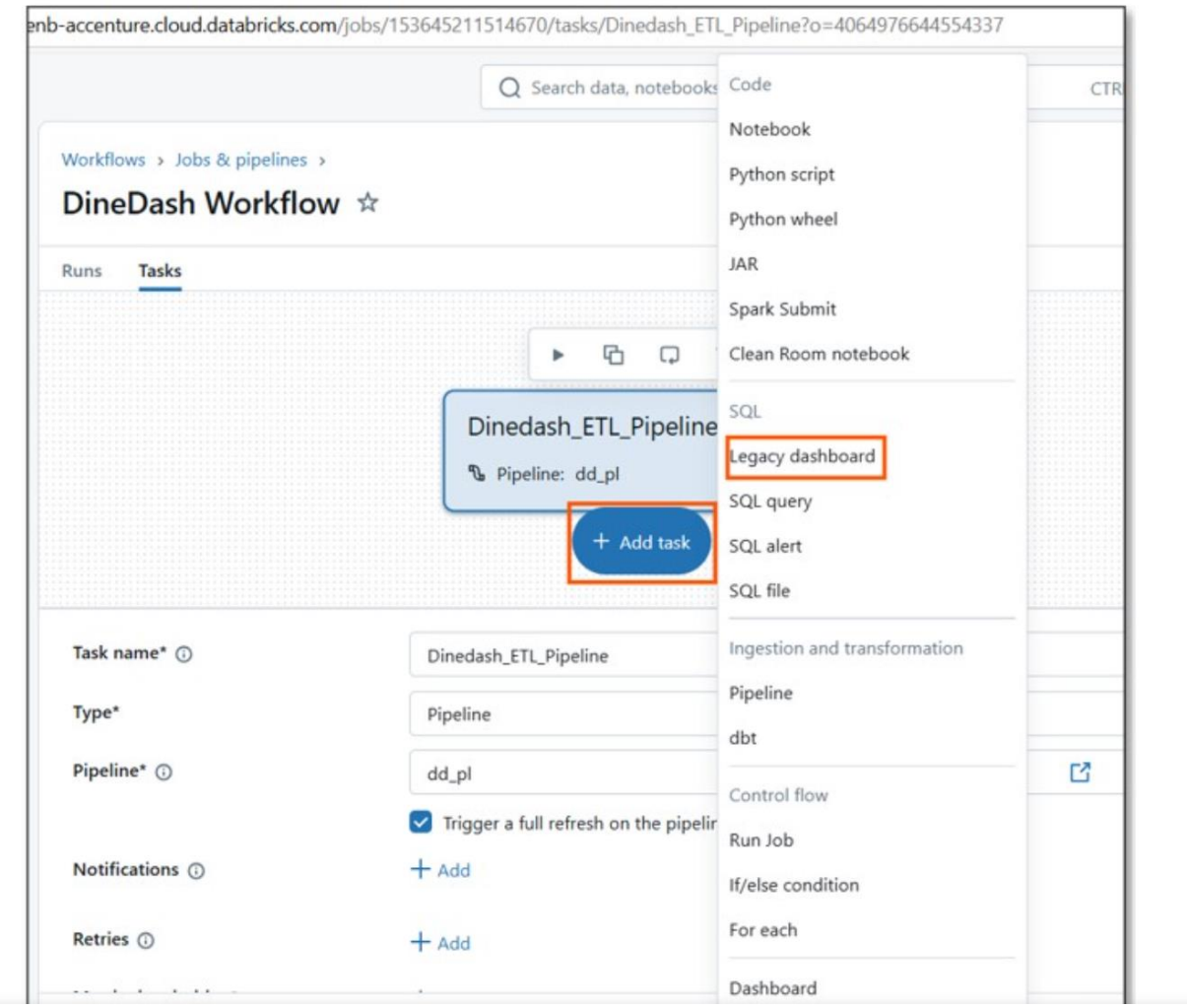


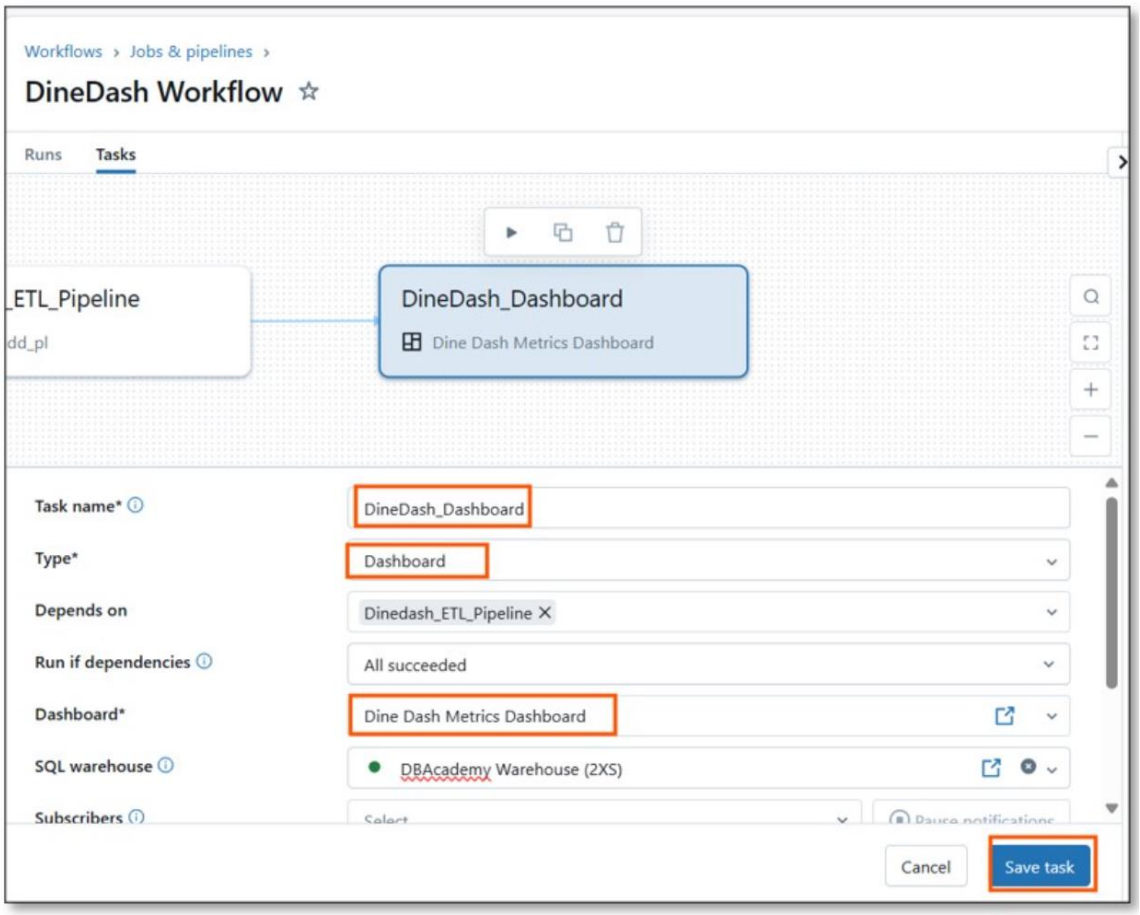


Provide name for your job and add Pipeline as the first task

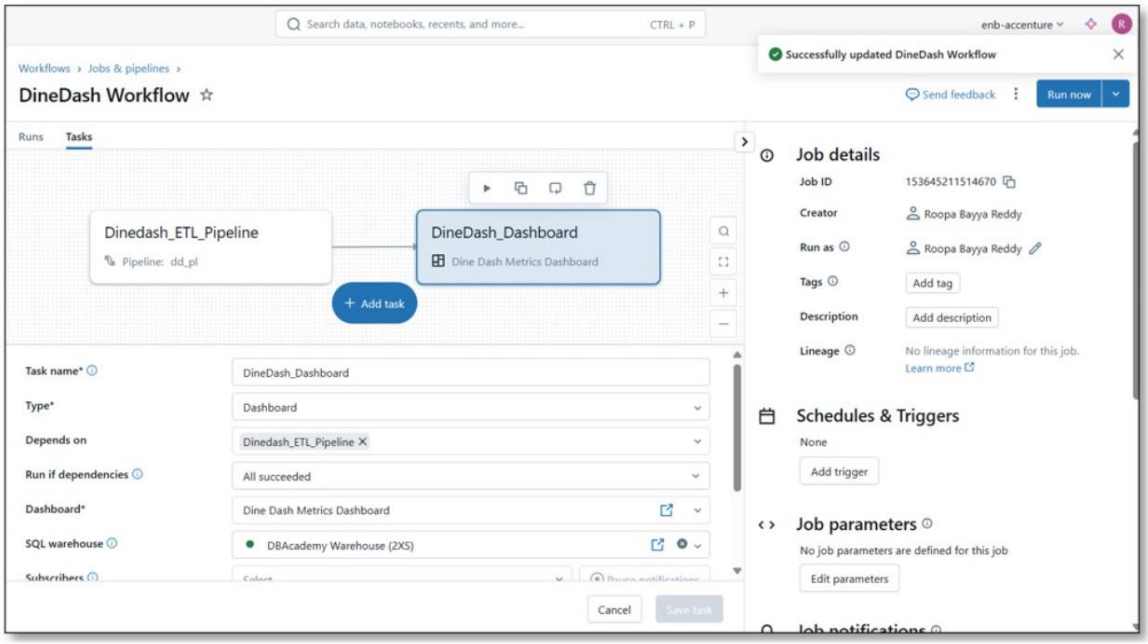


Click on Add Task and choose legacy dashboard





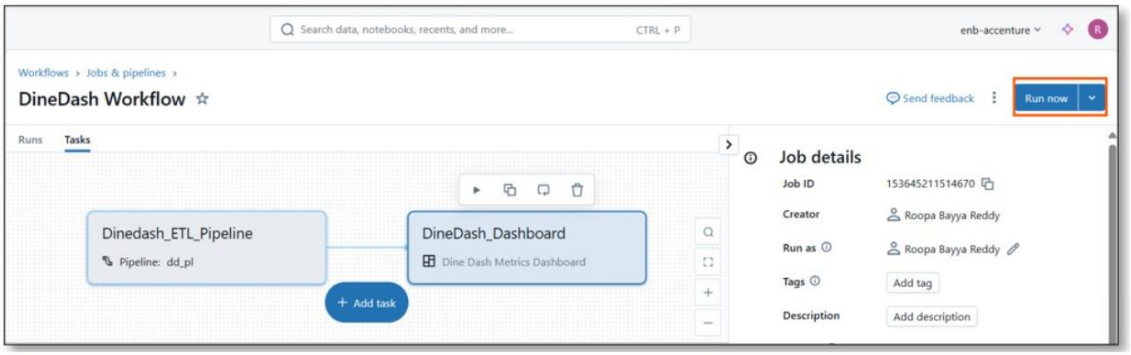
You completed workflow setup



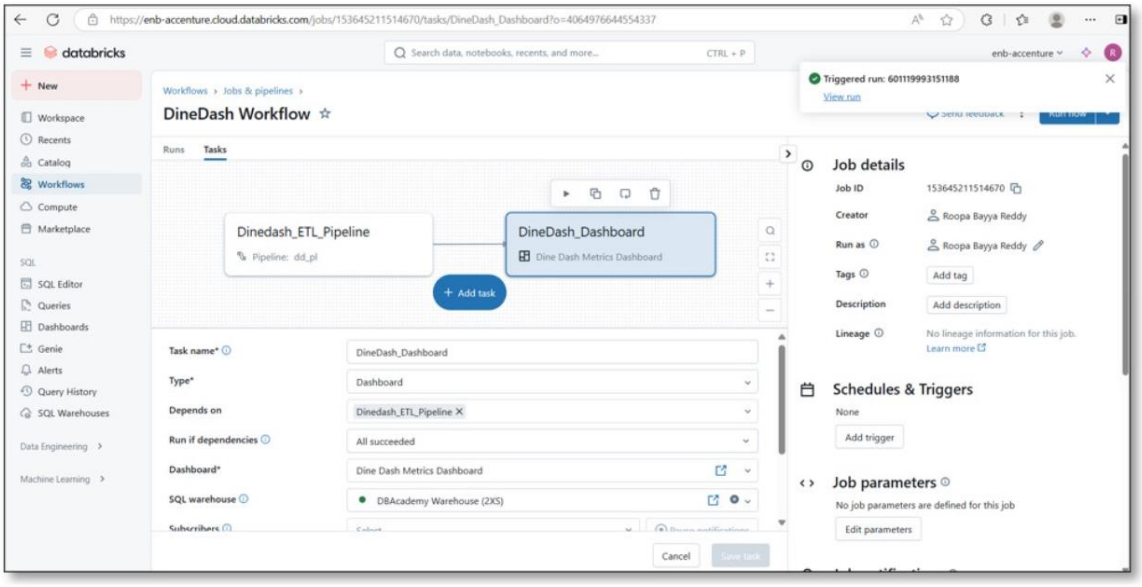
Just like for DLT execution learning, start with few sets of orders data files.

After first run, observe the results

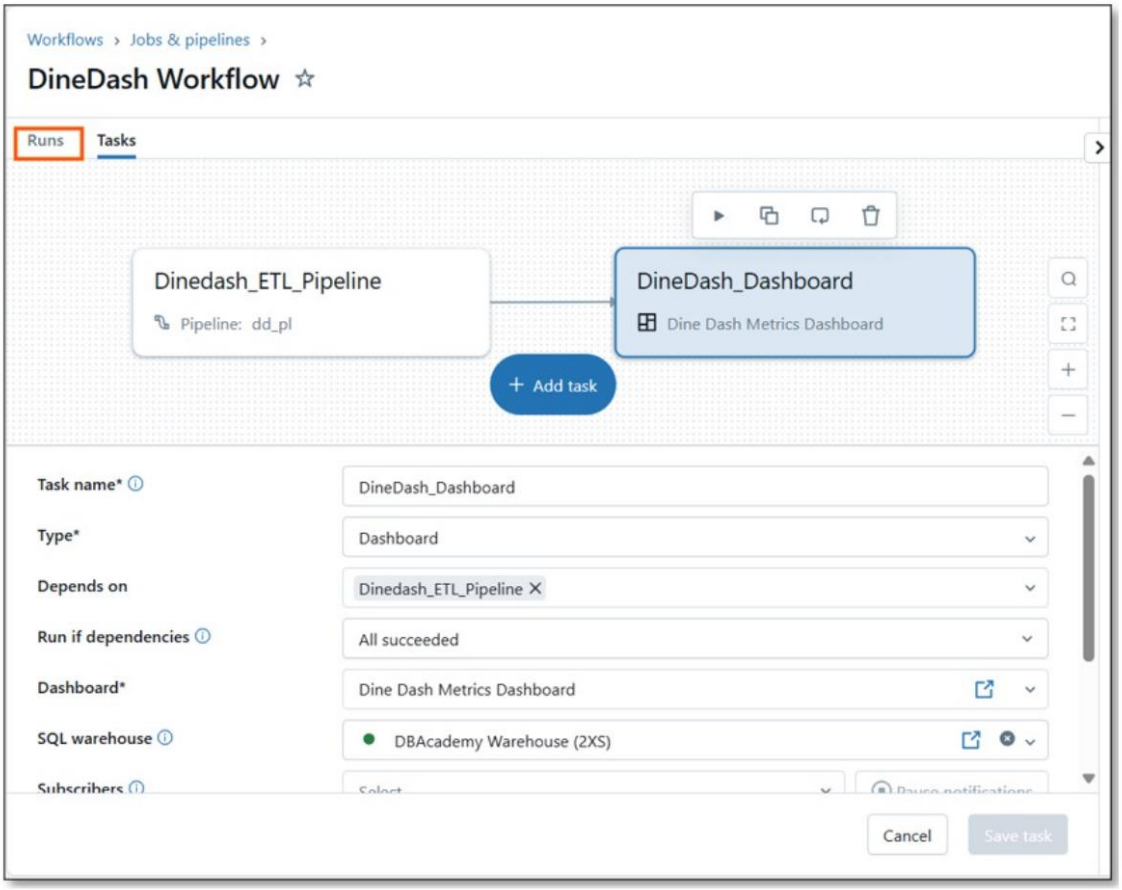
Execute the Job by clicking on Run Now



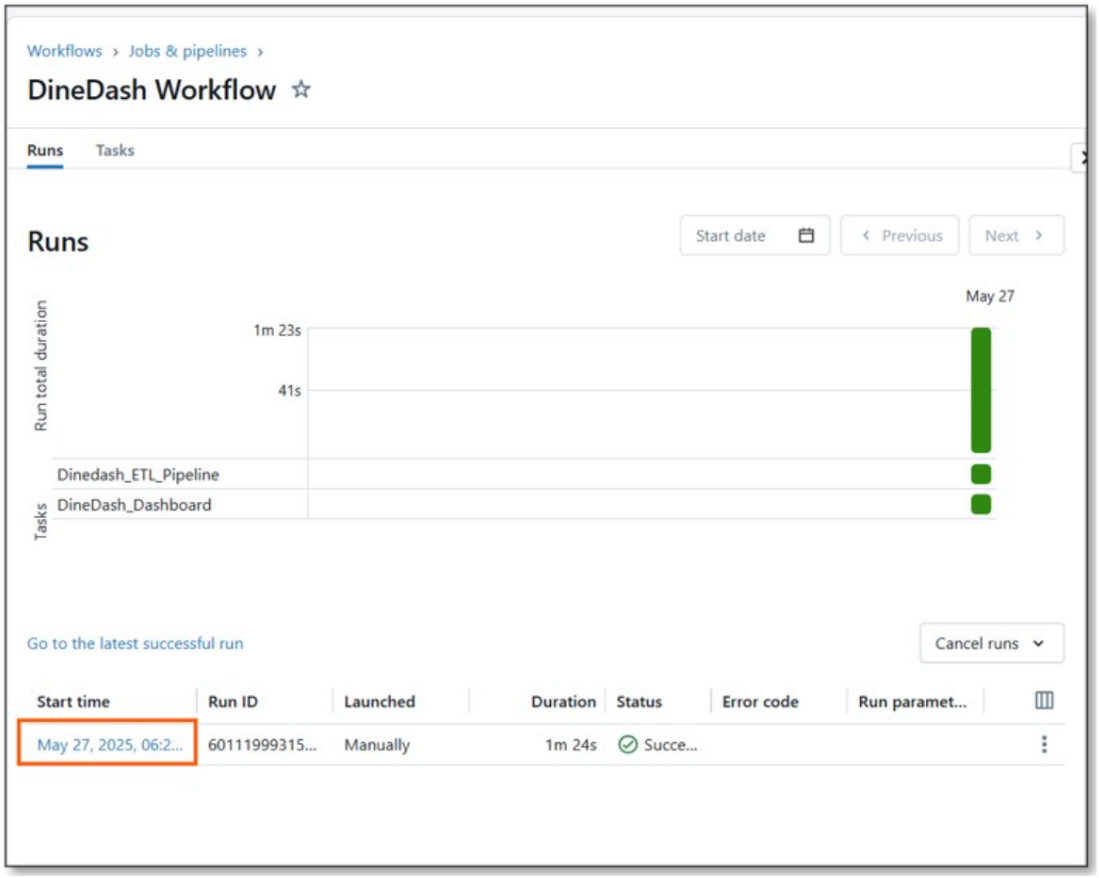
This created new Run ID for the current execution of the job



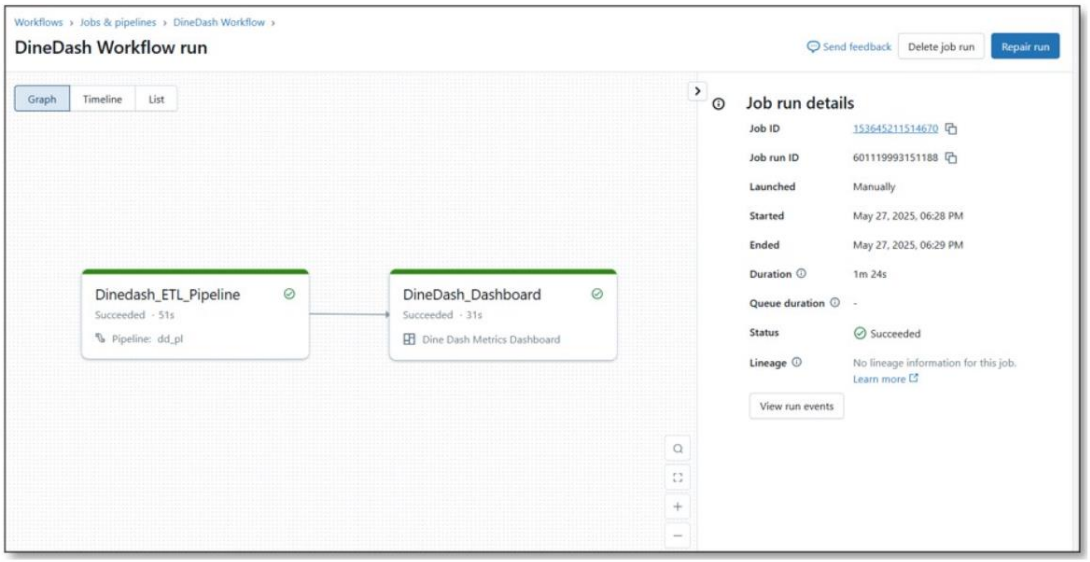
Click on Runs



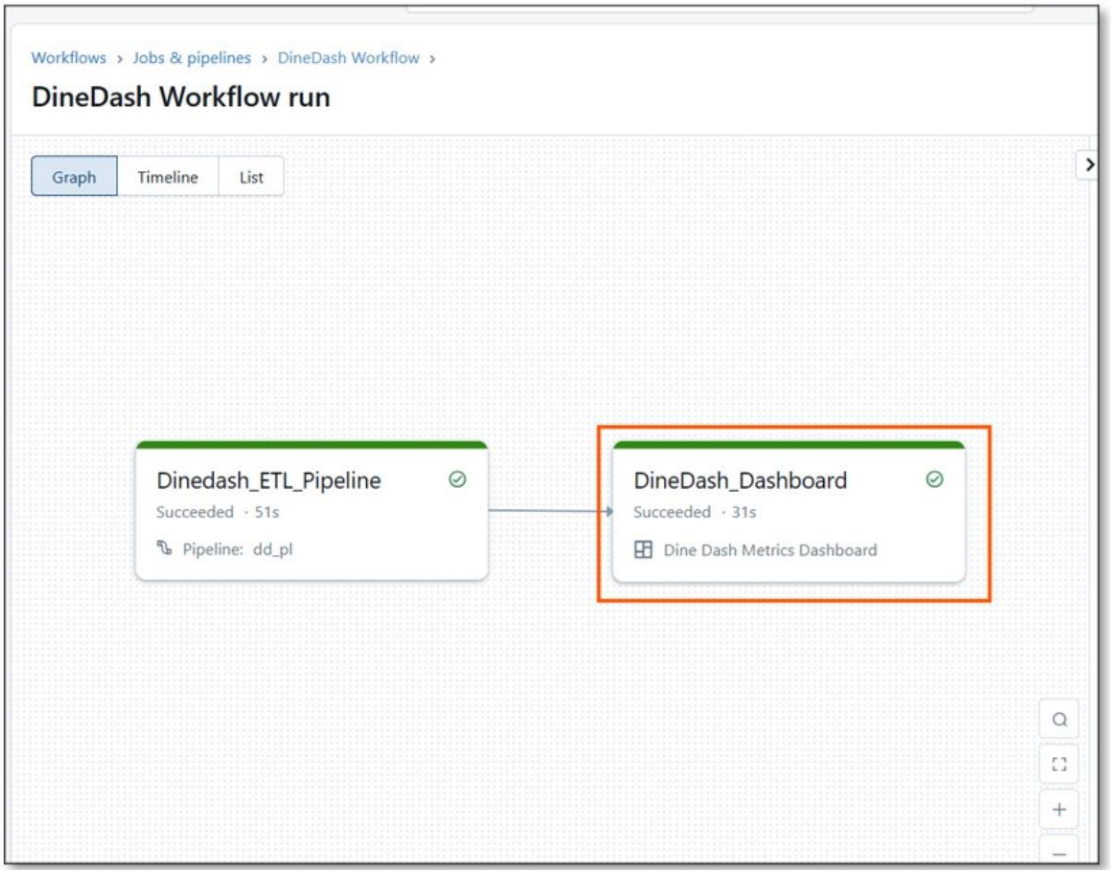
Click on the hyper link listed againt the current run



This displays the details of the current run execution

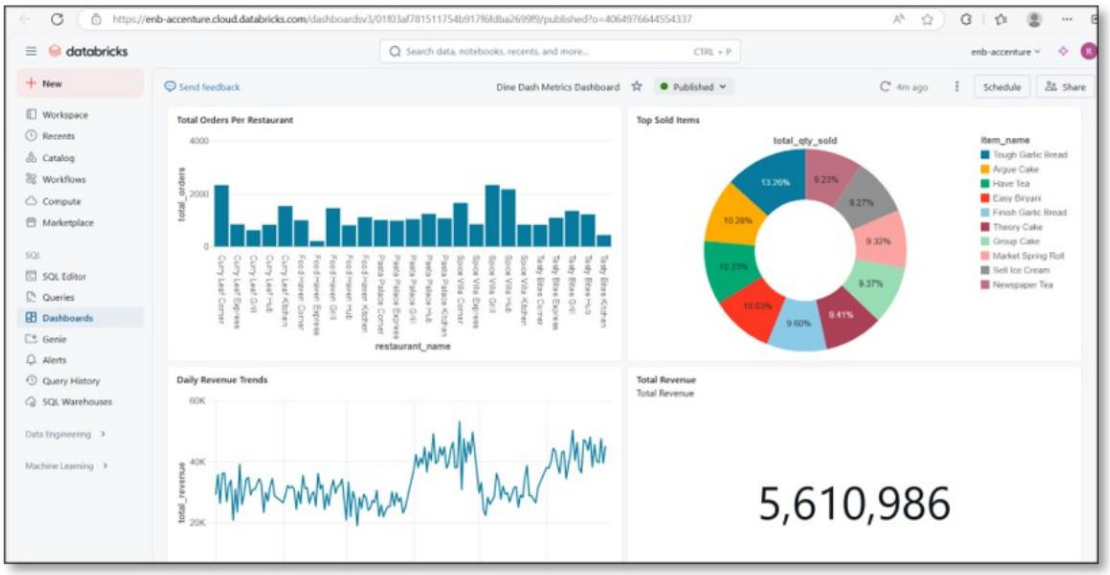


Click on Dashboard task



This gives link to the Dashboard you have cerated

Click on the Dashboard link and observe the metrics



Add few more orders datasets to the orders folder,Click on Run Now

Repeat the steps to observe the incremental data updates on to Dashboard